

AVPEU: Anonymous Verifiable Presentations with Extended Usability

Yalan Wang¹, Liqun Chen¹, Yangguang Tian¹, Long Meng¹, Christopher J.P. Newton¹

Abstract

The World Wide Web Consortium (W3C) has established standards for decentralized identities (DIDs) and verifiable credentials (VCs). A DID serves as a unique identifier for an entity, while a VC validates specific attributes associated with the DID holder. To prove ownership of credentials, users generate verifiable presentations (VPs). To enhance privacy, the W3C standards advocate for randomizable signatures in VC creation and zero-knowledge proofs for VP generation. However, these standards face a significant limitation: they cannot effectively verify cross-domain credentials while maintaining anonymity. In this paper, we present Anonymous Verifiable Presentations with Extended Usability (AVPEU), a novel framework that addresses this limitation through the introduction of a notary system. At the technical core of AVPEU lies our proposed randomizable message-hiding signature scheme. We provide both a generic construction of AVPEU and specific implementations based on Boneh-Boyen-Shacham (BBS), Camenisch-Lysyanskaya (CL), and Pointcheval-Sanders (PS) signature. Our experimental results demonstrate the feasibility of these schemes.

1 Introduction

Concerns about how individuals and organizations make use of personal information have led to research on an emerging method of identity management, called decentralized identity (DID) [14, 16, 26, 27]. The Decentralized Identifier Working Group of W3C [27] has been developing standards [28] for DID management, encompassing three key components: decentralized identifiers (DIDs), verifiable credentials (VCs), and verifiable presentations (VPs). In the W3C standards [28], a DID represents a verifiable and decentralized digital identity that serves as a novel identifier for its holder. Typically, an issuer creates a VC for the DID holder based on the holder's attributes, such as name, address, and date of birth. The DID holder can then demonstrate ownership of the VC through a VP.

The W3C standards outline the fundamental processes involving DIDs, VCs, and VPs. The process begins when a user (the DID holder) generates a key pair, with the verified public key serving as their DID. To obtain a VC, the DID holder submits a request to an issuer, including their relevant attributes and proof of DID ownership. The issuer verifies both

¹University of Surrey, {yw0010,liqun.chen, yangguang.tian, long.meng, c.newton}@surrey.ac.uk

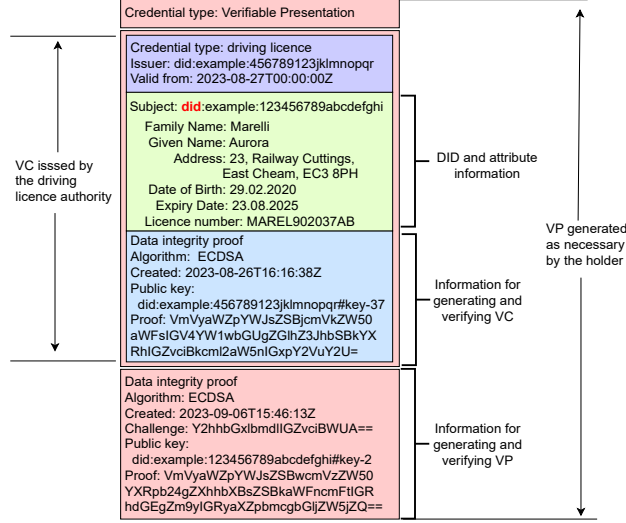


Figure 1: A VP for a driving licence.

the holder’s attributes and DID before issuing a VC. Subsequently, the holder can generate a VP using their secret key and associated attributes to demonstrate VC ownership to a verifier.

Anonymous VPs play an important role in practical applications, as illustrated by the driving license example in Figure 1. This VP associates with name, address, date of birth, expiry date and licence number. This VP contains multiple attributes: name, address, date of birth, expiry date, and license number. The structure comprises two key components: a blue section containing the signature algorithm for generating the Verifiable Credential (VC) and its corresponding verification public key, and a red section containing the signature algorithm for generating the VP and its associated verification public key. Beyond its primary function of validating driving privileges, this VP can serve various privacy-preserving purposes, such as anonymous age verification. In addition to the driving licence example shown in Figure 1, in real applications, VPs can be applied in many scenarios, such as EU identity card, birth certificates, academic degrees and bank statements. Further, in the W3C standards, there are a lot of use cases, such as education (digital transcript, taking a test, transferring schools, online classes), finance (reuse know your customer, money transfer).

In the W3C standards [28], they recommend randomizable signatures to create VCs and zero-knowledge proofs to generate anonymous VPs. Possible candidates for randomizable signatures can be Boneh-Boyen-Shacham (BBS) signature [3] or Camenisch-Lysyanskaya (CL) signature [8].

However, the W3C standards’ approach to anonymous VPs faces significant limitations in cross-domain applications. For instance, a driving license issued in the United States requires notarization by trusted

authorities before becoming valid in the United Kingdom. Similar challenges arise with other credentials such as education (digital transcript, taking a test, transferring schools, online classes), finance (reuse know your customer, money transfer), birth certificates, academic degrees, and bank statements. The fundamental issue lies in the cross-domain setting: anonymous VPs of VCs cannot be directly utilized across different domains because issuers operate independently and lack mutual trust. This raises a critical research question: *How can we ensure universal usability of anonymous VPs in cross-domain settings?*

Contributions. We propose a novel framework called Anonymous Verifiable Presentations with Extended Usability (AVPEU) to address the limitations inherent in the W3C recommended approach. Our generic construction introduces a notary, which can be implemented as either a single entity or multiple parties in a distributed configuration. For clarity of exposition, we present the framework with a single notary entity. The protocol operates as follows: When a DID holder presents a VC generated using either randomizable or non-randomizable signatures, the notary validates this credential against the original issuer’s public key. Upon successful verification, the notary generates a new credential derived from the DID holder’s public key using a randomizable signature scheme. Subsequently, the DID holder can leverage this notarized credential to generate anonymous VPs. A crucial privacy-preserving feature of our construction is that the notary remains blind to the DID holder’s attributes throughout the notarization process. The verification of these anonymous VPs relies on the notary’s public key rather than the original issuer’s key. To ensure transparency and auditability, all public information generated during the notarization process can be recorded in a public verifiable data registry.

We introduce a novel cryptographic primitive called Randomizable Message-Hiding Signature (RMHS), which serves as the fundamental building block for AVPEU. Messages in RMHS correspond to attributes in a VC/VP. We demonstrate that RMHS can be instantiated using three established signature schemes: CL signature [8], BBS signature [3], and Pointcheval-Sanders (PS) signature [23]. We present PS-based RMHS to showcase a concrete construction of AVPEU. Our contributions can be summarized as follows:

- We propose a generic construction of AVPEU. We introduce a trusted notary to ensure the universally verified anonymous VPs in the cross-domain settings. Notably, our construction removes the constraint that all VCs must be generated using randomizable signatures.
- We propose a new primitive RMHS, which is of independent interest. We provide concrete instantiations based on well-known randomizable signatures, CL signature [8], BBS signature [3] and Pointcheval-Sanders [23] (PS) signature.
- We implement three versions of AVPEU by integrating CL, BBS, and PS-based RMHS. The results confirmed their practicality. For example, in PS-based instantiation, the credential sizes is constant, about 0.37KB. When the number of attributes is 20 in the PS-based

instantiation, the running times for VP generation and verification are 0.09s and 0.06s, respectively.

The research of anonymous credentials (ACs) is diverse after the first anonymous credential scheme [7]. AC is also known as attribute-based anonymous credentials (ABCs) [6]. Briefly, a user owns a set of attributes and holds a credential associated with his/her attributes after credential issuance. Later, the user can selectively disclose some of attributes during a credential showing. We list a few closely related research areas. The delegatable credentials [1, 2] allow a credential holder to delegate his/her credential to a third party. Anyone should be able to verify the delegated credential's validity against the original issuer's public key. The issuer-hiding credentials [4, 25] aim to hide an (or multiple) issuer's public key. Anyone verifies the credential's validity against a set of public keys during a credential proof. Besides, several post-quantum group signatures based on ACs have been proposed [12, 9, 10, 11]. Compared to various AC schemes proposed in the literature, our pairing-based scheme distinguishes itself by specifically addressing the challenge of extending anonymous VP usability in cross-domain environments.

The rest of our paper is organized as follows. In Section 2, we introduce all building blocks. In Section 3, we first present a new message-hiding signature scheme and prove its security. Then, we present a generic construction of AVPEU, and a concrete construction. We implement three instances of AVPEU in Section 4, and give conclusion in Section 5.

2 Building Blocks

In this section, we introduce all building blocks for our AVPEU construction.

2.1 Digital Signatures

A digital signature scheme **SIG** involves two players: a signer and a verifier.

Definition 1 ***SIG** = (KeyGen, Sign, Verify) consists of three algorithms:*

- *KeyGen(λ) $\rightarrow$ (sk, pk): This key generation algorithm takes a security parameter λ as input and outputs a secret-public key pair (sk, pk) for the signer.*
- *Sign(sk, m) $\rightarrow$ σ : This signing algorithm takes the secret key sk and a message m as input and outputs a signature σ .*
- *Verify(pk, m, σ) $\rightarrow$ 0/1: This verification algorithm takes the public key pk , a message m and a signature σ as input and outputs "1" for accepting σ as a valid signature, "0" for rejecting.*

The security of a signature scheme **SIG** is captured by security properties of correctness and existential unforgeability under adaptive chosen-message attacks (EU-CMA). Definitions of these two properties are given as follows:

Experiment $\text{Exp}_{\mathcal{A}}^{\text{unforge}}(\lambda)$

- $(sk, pk) \leftarrow \text{KeyGen}(\lambda)$
- $(M^*, \sigma^*) \leftarrow A^{\text{Sign}(sk, \cdot)}(pk)$.
- Return 1 iff $\text{Verify}(pk, M^*, \sigma^*) = 1 \wedge M^* \notin \mathcal{M}$

Figure 2: The unforgeability (EU-CMA) experiment.

Definition 2 (*Correctness*) A digital signature scheme, **SIG**, is correct, if $\text{KeyGen}(\lambda) \rightarrow (sk, pk)$, $\text{Sign}(sk, m) \rightarrow \sigma$, then it guarantees $\text{Verify}(pk, m, \sigma) = 1$.

Unforgeability. The unforgeability of **SIG** is defined using an experiment $\text{Exp}_{\mathcal{A}}^{\text{unforge}}$, which is played between a simulator $\mathcal{S}$ and an adversary $\mathcal{A}$. It is shown in Figure. 2, where $\mathcal{M}$ is the message space in which each message has been used by $\mathcal{A}$ to query to the signing oracle **Sign**. The signing oracle **Sign** is defined that given a message as input, it returns a corresponding signature. We denote the advantage of $\mathcal{A}$ as $\text{adv}_{\mathcal{A}}^{\text{unforge}}(\lambda)$, which is defined by the probability that $\mathcal{A}$ wins the unforgeability game.

Definition 3 (*Unforgeability*) A signature scheme **SIG** is said to be unforgeable if for any polynomial time adversary $\mathcal{A}$, the following equation holds:

$$\text{adv}_{\mathcal{A}}^{\text{unforge}}(\lambda) = \Pr \left[\text{Exp}_{\mathcal{A}}^{\text{unforge}}(\lambda) = 1 \right] \leq \text{negl}(\lambda)$$

2.2 Randomizable Signatures

A randomizable signature scheme [3, 8, 23] is a digital signature scheme with a property that signatures can be randomized.

Definition 4 A randomizable signature scheme denoted as **rSIG**, $\text{rSIG} = (\text{KeyGen}, \text{Sign}, \text{rSign}, \text{Verify})$ consists of four algorithms as follows:

- $\text{KeyGen}(\lambda) \rightarrow (sk, pk)$: The same as **SIG**.KeyGen.
- $\text{Sign}(sk, m) \rightarrow \sigma$: The same as **SIG**.Sign.
- $\text{rSign}(\sigma, m) \rightarrow \sigma'$: This randomizing signing algorithm takes a signature σ and a message m as input, outputs another signature σ' on m .
- $\text{Verify}(pk, m, \{\sigma, \sigma'\}) \rightarrow 0/1$: The same as **SIG**.Verify.

The security of a randomizable signature scheme **rSIG** is also captured by security properties of correctness and unforgeability as the same as **SIG**.

2.3 Randomizable Message-hiding Signatures

In this subsection, we introduce randomizable message-hiding signature (RMHS). In a RMHS, multiple messages (a message vector, the size of which is L), acting as multiple attributes in AVPEU, are hidden from

the public. A RMHS scheme involves three types of players: a signer, a signature holder, and a verifier. Compared with a traditional digital signature scheme, a RMHS does not reveal the signed messages to a verifier while still convincing the verifier that a signature is correctly created under a given public key. In a RMHS scheme, a non-interactive zero-knowledge proof (NIZKP) is required, which is introduced in Appendix 2.4.

Definition 5 *A randomizable message-hiding signature scheme, which is denoted as $\mathbf{RMHS} = (\text{KeyGen}, \text{Sign}, \text{Verify}, \text{hSign}, \text{hVerify})$, consists of following five algorithms in detail:*

- $\text{KeyGen}(\lambda) \rightarrow (sk, pk)$: The same as $\mathbf{rSIG.KeyGen}$.
- $\text{Sign}(sk, \mathbf{m}) \rightarrow \sigma$: The same as $\mathbf{rSIG.Sign}$ run by the signer.
- $\text{Verify}(pk, \mathbf{m}, \sigma) \rightarrow 0/1$: The same as $\mathbf{rSIG.Verify}$.
- $\text{hSign}(\sigma, \mathbf{m}) \rightarrow \sigma'$: This message-hiding signing algorithm takes a signature σ and a message vector $\mathbf{m}$ as input, outputs another signature σ' on $\mathbf{m}$. Note that this algorithm is performed by a signature holder.
- $\text{hVerify}(pk, \sigma') \rightarrow 0/1$: This message-hiding verification algorithm takes (pk, σ') as input, outputs 0/1 for rejecting/accepting the signature.

The security of a RMHS scheme is captured by security properties of correctness, message-hiding and unforgeability.

Correctness. Informally, it means signatures generated by honest signers and honest signature holders are always valid by the verification algorithm.

Definition 6 (Correctness) *A RMHS scheme is correct that we can get the result $1 \leftarrow \text{hVerify}(pk, \sigma')$ and $1 \leftarrow \text{Verify}(pk, \mathbf{m}, \sigma)$ if $\sigma' \leftarrow \text{hSign}(\sigma, \mathbf{m})$, $\sigma \leftarrow \text{Sign}(sk, \mathbf{m})$, where $(sk, pk) \leftarrow \text{KeyGen}(\lambda)$.*

Message-hiding. Informally, given a signature, (optionally) the secret key used to generate the signature and two message vectors chosen by an adversary $\mathcal{A}$, $\mathcal{A}$ cannot decide which message vector has been used in generating the signature. Especially, we allow $\mathcal{A}$ to access the signer's signing key, *not* the internal randomness used in generating the signature. The message-hiding experiment $\text{Exp}_{\mathcal{A}}^{\text{m-hide}}(\lambda)$ is performed between an adversary $\mathcal{A}$ and a simulator $\mathcal{S}$ as follows.

- **KeyGen:** $\mathcal{S}$ runs the $\mathbf{RMHS.KeyGen}$ algorithm to obtain key pairs for all signers. Then $\mathcal{S}$ sends all signers' public keys to $\mathcal{A}$. $\mathcal{M}$ is the message space and $\mathcal{Q}$ is the uncorrupted holder set.
- **Queries:** $\mathcal{A}$ may issue the following queries adaptively.
 - **Signing.** $\mathcal{A}$ issues a signing query for a public key pk and a message vector $\mathbf{m}$, $\mathcal{S}$ returns a signature $\sigma \leftarrow \mathbf{RMHS.Sign}(sk, \mathbf{m})$ to $\mathcal{A}$.
 - **hSigning.** $\mathcal{A}$ issues a message-hidden signing query for a message vector $\mathbf{m}$, $\mathcal{S}$ returns a signature $\sigma' \leftarrow \mathbf{RMHS.hSign}(\sigma, \mathbf{m})$ to $\mathcal{A}$.
 - **Corrupt.** $\mathcal{A}$ issues a corrupt query for pk , $\mathcal{S}$ returns a secret key sk to $\mathcal{A}$, the pk and corresponding signer will be removed from $\mathcal{Q}$.
- **Challenge:** $\mathcal{A}$ submits a signer's public key $pk^* \in \mathcal{Q}$ and two message vectors $(\mathbf{m}_0, \mathbf{m}_1)$ from $\mathcal{M}$ to $\mathcal{S}$, $\mathcal{S}$ signs one of the vector $\mathbf{m}_b$ for $b \in \{0, 1\}$ and returns the corresponding signature σ_b^* to $\mathcal{A}$ and removes pk^* from $\mathcal{Q}$. $\mathcal{A}$ may continue issuing corrupt queries for pk^* to get the

corresponding signing key sk^* . Finally, $\mathcal{A}$ outputs b^* as its guess. If $b^* = b$, $\mathcal{S}$ outputs 1; Otherwise, $\mathcal{S}$ outputs 0. We define the advantage of $\mathcal{A}$ to win $\text{Exp}_{\mathcal{A}}^{\text{m-hide}}(\lambda)$ as

$$\text{Adv}_{\mathcal{A}}^{\text{m-hide}}(\lambda) = |\Pr[\text{Exp}_{\mathcal{A}}^{\text{m-hide}}(\lambda) = 1] - 1/2|.$$

Definition 7 (*Message-hidding*) A RMHS scheme ensures message-hiding if for any PPT $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{\text{m-hide}}(\lambda)$ is negligible in λ .

Unforgeability. Informally, an adversary without the knowledge of signing keys, cannot generate a valid signature. The unforgeability experiment $\text{Exp}_{\mathcal{A}}^{\text{Unforge}}(\lambda)$ is performed between an adversary $\mathcal{A}$ and a simulator $\mathcal{S}$ as follows:

- **KeyGen:** $\mathcal{S}$ runs the **RMHS.KeyGen** algorithm to obtain signers' key pairs. $\mathcal{M}$ is a message space and $\mathcal{Q}$ is the uncorrupted signer set.
- **Queries:** Same as the queries defined in the message-hiding experiment.
- **Forgery:** $\mathcal{A}$ outputs a signature σ'^* with pk^* and $\mathcal{A}$ wins if
 - **RMHS.hVerify**(pk^*, σ'^*) = 1;
 - σ'^* has been returned in previous hSigning queries.

We define the advantage of $\mathcal{A}$ as

$$\text{Adv}_{\mathcal{A}}^{\text{Unforge}}(\lambda) = |\Pr[\mathcal{A} \text{ wins}]|.$$

Definition 8 (*Unforgeability*) A RMHS scheme is unforgeable if for any PPT $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{\text{Unforge}}(\lambda)$ is negligible in λ .

2.4 Non-Interactive Zero-Knowledge Proof (NIZKP)

Let $\mathcal{L} = \{e | \exists w : \mathcal{R}(e, w) = 1\}$ be an NP-language with a relation $\mathcal{R}$. A Non-Interactive Zero-Knowledge Proof (NIZKP) allows a prover to prove the statement e in the language $\mathcal{L}$. The formal definition is shown below.

Definition 9 An **NIZKP** Π for language $\mathcal{L}$ includes the following three algorithms.

- **Setup**(1^λ) $\rightarrow$ crs_Π : This setup algorithm takes a security parameter λ as input, outputs a public common reference string crs_Π .
- **Proof**(crs_Π, e, w) $\rightarrow$ π : This proof algorithm takes the crs_Π , a statement e and a witness w as input, outputs a proof π .
- **Verify**(crs_Π, e, π) $\rightarrow$ 0/1: This verification algorithm takes the crs_Π , a statement e and a proof π as input, outputs a bit 1 for “accept” that the statement is valid, or 0 for “reject”, the statement is invalid.

An **NIZKP** typically satisfies three properties: correctness, zero-knowledge, and soundness. Correctness (or completeness) says that an honest prover should always be able to convince the verifier that $e \in \mathcal{L}$. Zero-knowledge says that the receiver of the proof π cannot learn anything except the validity of the statement. Soundness says that a malicious prover cannot

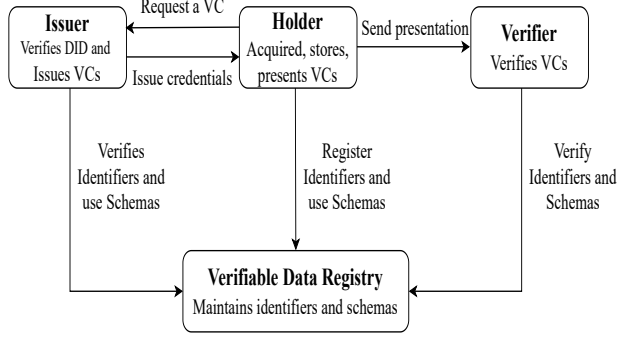


Figure 3: The framework of a DID and VC system [28]

generate a proof π for any element $e \notin \mathcal{L}$. We use a stronger formulation of the soundness called simulation-sound extractability [13] in this work. It says that every adversary who can generate a proof π^* for a statement e^* must know the witness w^* , even when seeing simulated proofs for adaptively chosen statements not in language $\mathcal{L}$. The detailed definition of three properties is put in Appendix A.

2.5 Overview of W3C DID and VC standards

In a DID system, we regard a DID holder and a subject as one entity, i.e., a holder. There are four types of entities:

- **Holder:** An entity, with a DID, who possesses one or more VCs and generates VPs from them.
- **Issuer:** An entity who confirms the DID and attributes about a holder, creates a VC based on the DID and these attributes, and sends the VC to the holder.
- **Verifier:** An entity who receives one or more VCs, optionally inside a VP, for verification.
- **Verifiable data registry:** A system that mediates the creation and verification of identifiers, keys and other relevant data and ensures that it is tamper-resistant.

Decentralized Identifiers (DIDs). A DID [5, 17, 18, 19, 20, 21, 27] is a new type of universal resource identifier (URI) [22], which can be resolved to a DID document that includes cryptographic material, verification methods, and service endpoints, but keeps a DID holder’s private information hidden. The DID and its corresponding DID document provide a verifiable and decentralized approach to interact with others.

Verifiable Credentials (VCs). A VC is a tamper-evident credential with authorship that can be cryptographically verified [15]. To achieve

- $\text{KeyGen}(\lambda) \rightarrow (sk_i, pk_i)$: Given a security parameter λ , a secret-public key pair (sk_i, pk_i) for the issuer is generated by running $\mathbf{SIG.KeyGen}(\lambda)$.
- $\text{VC.Gen}\{\lambda; sk_i, pk_i\} \rightarrow \{did_h, \mathbf{att}_h; cre_{h,i}\}$: This is a VC generation protocol run between a holder h and an issuer i .
 1. h , with the did_h , requests a VC from the issuer i .
 2. i verifies h . If the verification fails, aborts. Otherwise, i issues a VC $cre_{h,i}$ by running $\mathbf{SIG.Sign}$, $cre_{h,i} \leftarrow \mathbf{SIG.Sign}(sk_i, (did_h, \mathbf{att}_h))$, where $\mathbf{att}_h$ is a set of attributes generated by the issuer for h .
 3. h verifies $cre_{h,i}$ by running $\mathbf{SIG.Verify}(pk_i, (did_h, \mathbf{att}_h), cre_{h,i})$. If the verification fails, aborts, otherwise, accepts $cre_{h,i}$.
- $\text{VP.Gen}((did_h, \mathbf{att}_h), cre_{h,i}) \rightarrow vp_h$: This is a VP generation algorithm run by a holder h . h generates a VP by running $\mathbf{SIG.Sign}$, $vp_h \leftarrow \mathbf{SIG.Sign}(did_h, \mathbf{att}_h, cre_{h,i})$. Then sends vp_h to the verifier.
- $\text{VP.Verify}(pk_i, cre_{h,i}, vp_h) \rightarrow 0/1$: This is a VP verification algorithm run by the verifier, who takes input $(pk_i, cre_{h,i}, vp_h)$ and runs $\mathbf{SIG.Verify}(pk_i, cre_{h,i}, vp_h)$. Outputs "1" for accepting vp_h or "0" for rejecting vp_h .

Figure 4: The framework of a DID and VC system without anonymity

cryptographic verification, signatures based on credentials can be generated according to the standards [28]. Rather than simply sign a byte-string, VC presents an abstract data model for the idea of *claims*, which are any list of attributes and values pertaining to a holder. The process of verifiable credential presentations is called verifiable presentations (VPs). The framework of a DID and VC system is shown in Figure 3 (derived from [28]). Making use of the digital signature algorithm $\mathbf{SIG}$, algorithms/protocols of a DID and VC system without anonymity are shown in Figure. 4.

3 Our Constructions

In various applications, anonymous VPs are required in real applications mentioned in introduction, such as the driving licence, EU identity card. The W3C standards recommends using randomizable signatures to generate VCs and zero-knowledge proofs to achieve anonymous VPs. However, the mechanism to achieve anonymous VPs recommended by the W3C standards is limited. Anonymous VPs cannot be directly verified before some authorities notarize them. To overcome the limitation, we introduce a notary to the DID and VC system. Given any DID holder's credential generated by a (non)-randomizable signature algorithm, the notary notarizes the credential and translates it into a new credential using

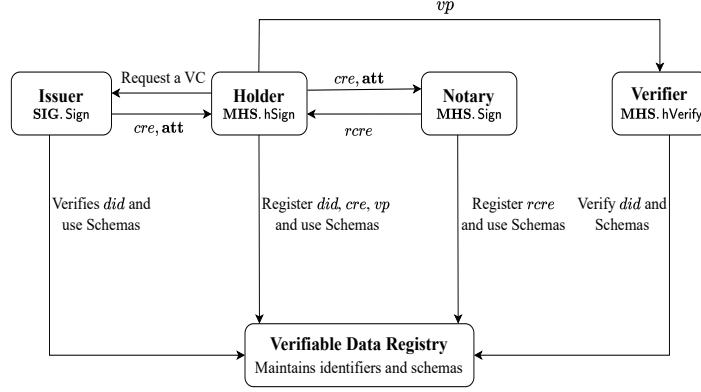


Figure 5: The framework of our generic construction

a randomizable signature algorithm. During verifications of anonymous VPs, the notary’s public key instead of an individual issuer’s public key is used. This solution ensures universal usability of anonymous credentials in cross domain settings. In the following subsections, we first introduce our proposed randomizable message-hiding signatures (RMHS). Then, we present our proposed generic construction and a concrete construction.

3.1 Randomizable Message-Hiding Signatures (RMHS)

In this subsection, we propose a concrete scheme based on the PS signature scheme and prove its security. This RMHS scheme will be used as a building block for our concrete construction, which will be introduced in Section 3.3, in which these messages play the role of hidden attributes.

3.1.1 The Original PS signature Scheme.

Here we introduce the multiple-message PS signature scheme [23] firstly. We define $[L] = \{1, \dots, L\}$ and let $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$ be three cyclic groups of prime order p and $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, a type III bilinear map. Here, for any group $\mathbb{X}$ ($\mathbb{G}_1$ or $\mathbb{G}_2$), $\mathbb{X}^* = \mathbb{X} \setminus \{1_{\mathbb{X}}\}$.

- **KeyGen**(λ) $\rightarrow (sk, pk)$: This key generation algorithm is run by the signer. Firstly, the signer generates the public parameters $pp \leftarrow (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$. Selects $g_2 \leftarrow_R \mathbb{G}_2^*$, $(x, \{y_i\}) \leftarrow_R \mathbb{Z}_p^{L+1}$, $i \in [L]$, computes $(X, \{Y_i\}) \leftarrow (g_2^x, \{g_2^{y_i}\})$. Sets sk as $(x, \{y_i\})$ and pk as $(pp, g_2, X, \{Y_i\})$, $i \in [L]$.
- **Sign**($sk, \mathbf{m}$) $\rightarrow \sigma$: This signing algorithm is run by the signer, where $\mathbf{m} = m_1, \dots, m_L$. Selects a random $g_1 \leftarrow_R \mathbb{G}_1^*$ and outputs $\sigma \leftarrow (g_1, g_1^{x + \sum_{i=1}^L y_i \cdot m_i})$.
- **Verify**($pk, \mathbf{m}, \sigma$) $\rightarrow 0/1$: This verification algorithm is run by the verifier. The verifier parses σ as (σ_1, σ_2) , checks whether $\sigma_1 \neq 1_{\mathbb{G}_1}$ and

$e(\sigma_1, X \cdot \prod Y_i^{m_i}) = e(\sigma_2, g_2)$ are both satisfied or not. If yes, outputs 1, and 0 otherwise.

The PS signature offers correctness and unforgeability, which has been proved in [23]. Note that we need the assumption 1 in [23] in our proof, which is described as follows:

Definition 10 (Assumption 1). Let $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$ be a bilinear group setting of type 3, with g (resp. $\tilde{g}$) a generator of $\mathbb{G}_1$ (resp. $\mathbb{G}_2$). For $(X = g^x, Y = g^y)$ and $(\tilde{X} = \tilde{g}^x, \tilde{Y} = \tilde{g}^y)$, where x and y are random scalars in $\mathbb{Z}_p$, we define the oracle $\mathcal{O}(m)$ on input $m \in \mathbb{Z}_p$ that chooses a random $h \in \mathbb{G}_1$ and outputs the pair $P = (h, h^{x+my})$. Given $(g, Y, \tilde{g}, \tilde{X}, \tilde{Y})$ and unlimited access to this oracle, no adversary can efficiently generate such a pair, with $h \neq 1_{\mathbb{G}_1}$, for a new scalar m^* , not asked to $\mathcal{O}$.

3.1.2 The Proposed RMHS.

Our RMHS scheme is an extension of the PS signature scheme and consists of following algorithms. Note that **KeyGen**, **Sign** and **Verify** are defined the same as the PS Scheme, so we only introduce the **hSign** and **hVerify** algorithms as follows:

- **hSign** $(\sigma, \mathbf{m}) \rightarrow \sigma'$: Parses $\mathbf{m} = \{m_1, \dots, m_L\}$, the signer chooses $r_1, r_2 \in_R \mathbb{Z}_p^*$ at random and compute $\sigma' = (S, \pi_{\sigma'})$, where $S = (s_1, s_2, s_3)$ and

$$s_1 = g_1^{r_1}, s_2 = g_1^{(r_2 + x + \sum y_i \cdot m_i) \cdot r_1}, s_3 = g_2^{r_2} \cdot \prod_{i \in [L]} Y_i^{m_i}$$

$$\pi_{\sigma'} = \mathbf{NIZKP}\{(pp, \{Y_i\}_{i \in [L]}, s_3); (r_2, \forall_{i \in [L]} m_i) | s_3 = g_2^{r_2} \cdot \prod_{i \in [L]} Y_i^{m_i}\}$$

- **hVerify** $(pk, \sigma') \rightarrow 0/1$: To verify σ' , the verifier first parses σ' as $(s_1, s_2, s_3, \pi_{\sigma'})$, then checks $s_i \neq 1_{\mathbb{G}_1}$ for $i = 1, 2$, the correctness of $\pi_{\sigma'}$, and $e(s_2, g_2) = e(s_1, X \cdot s_3)$. If all the checks pass, outputs 1 for “accept” the signature; otherwise, outputs 0 for “reject” the signature.

3.1.3 Security Analysis.

The security analysis includes correctness, message-hiding and unforgeability.

Theorem 1 The RMHS scheme in Section 3.1.2 is correct if the PS signature is correct and the **NIZKP** $\pi_{\sigma'}$ is correct.

Proof 1 Given a PS signature σ , it can pass the verification **RMHS.Sign** due to the correctness of the PS signature. Given the signature $\sigma' = (s_1, s_2, s_3, \pi_{\sigma'})$, pp and pk , where

$$s_1 = g_1^{r_1}, s_2 = g_1^{(r_2 + x + \sum y_i \cdot m_i) \cdot r_1}, s_3 = g_2^{r_2} \cdot \prod_{i \in [L]} Y_i^{m_i}$$

$$\pi = \mathbf{NIZKP}\{(pp, \{Y_i\}_{i \in [L]}, s_3); (r_2, \forall_{i \in [L]} m_i) | s_3 = g_2^{r_2} \cdot \prod_{i \in [L]} Y_i^{m_i}\}$$

we can compute as follows:

$$\begin{aligned}
e(s_2, g_2) &= e(g_1^{(r_2+x+\sum y_i \cdot m_i) \cdot r_1}, g_2) \\
&= e(g_1^{r_1}, g_2^x \cdot g_2^{r_2+\sum y_i \cdot m_i}) \\
&= e(s_1, X \cdot s_3)
\end{aligned}$$

And due to the correctness of $\pi_{\sigma'}$, $\pi_{\sigma'}$ generated on s_3 can pass the verification. Therefore, the proposed RMHS scheme is correct.

Theorem 2 The RMHS scheme in Section 3.1.2 offers message-hiding if $S = (s_1, s_2, s_3)$ meets the definition of indistinguishability (IND), and the **NIZKP** $\pi_{\sigma'}$ is zero-knowledge.

Proof 2 We define a sequence of games Game_i , $i \in [0, 3]$ and let Adv_i denote the advantage of the adversary in game Game_i . We consider each message-hidden signing query with a single message below (i.e., $L = 1$). The proof for the single message case can be easily extended to the multiple messages case. We assume that $\mathcal{A}$ issues at most q signing queries at each game.

- Game_0 : This is the original game for the message-hiding.
- Game_1 : This game is identical to game Game_0 except that $\mathcal{S}$ randomly chooses $g \in [1, q]$ as a guess for the index of the challenge query with respect to a challenge holder pk_0 . $\mathcal{S}$ will output a random bit if $\mathcal{A}$'s challenge query does not occur in this g -th query. Since at most n signers and q queries involved in the game, we have

$$\text{Adv}_0 = n \cdot q \cdot \text{Adv}_1$$

- Game_2 : This game is identical to game Game_1 except that $\mathcal{S}$ outputs a random bit if a **Guess**₁ event occurs in the g -th query. Here the **Guess**₁ event means the probability of $b' = b$ for guessing $S = (s_1, s_2, s_3)$ correctly. Then we have:

$$|\text{Adv}_1 - \text{Adv}_2| \leq \Pr[\text{Guess}_1]$$

Lemma 1 The **Guess**₁ event happens only with negligible probability when S meets the definition of IND.

Let $\mathcal{S}$ denote a simulator, choosing a random bit $b \in [0, 1]$ and aiming to play an IND game with $\mathcal{A}$. The process is as follows.

- $\mathcal{S}$ sets up the game for $\mathcal{A}$ by creating n signers with the corresponding secret and public key pairs. Also, $\mathcal{S}$ creates common random strings crs and trapdoor information τ for the **NIZKP** and an empty set $\mathcal{Q}$.
- During queries stage, $\mathcal{S}$ can simulate signing and corrupt queries easily because all signing keys and trapdoor information are known to $\mathcal{S}$.
- At the challenge stage, if $\mathcal{A}$ submits a challenge $pk \in \mathcal{Q}$ and two messages (m_0, m_1) . Then, $\mathcal{S}$ simulates a signature $\sigma' = (s_1, s_2, s_3, \pi_{\sigma'})$ based on m_b , where $s_1 = g_1^{r_1}$, $s_2 = g_1^{(r_2+x+y \cdot m_b) \cdot r_1}$, $s_3 = g_2^{r_2} \cdot Y^{m_b}$.

Note that after corrupt queries to pk , $\mathcal{A}$ knows (x, y, m_0, m_1) . Then $\mathcal{A}$ can compute s'_2 using x :

$$s'_2 = s_2/s_1^x = (g_1^{r_1})^{r_2+y m_b}$$

Then, using (y, m_0, m_1) , computes as follows:

$$s'_{2,0} = (g_1^{r_1})^{r_2+y(m_b-m_0)}, \quad s_{3,0} = g_2^{r_2+y(m_b-m_0)}$$

$$s'_{2,1} = (g_1^{r_1})^{r_2+y(m_b-m_1)}, \quad s_{3,1} = g_2^{r_2+y(m_b-m_1)}$$

No matter $(s_1, s'_{2,0}, s_{3,0})$ or $(s_1, s'_{2,1}, s_{3,1})$, two corresponding pairing checks $e(s'_{2,0}, g_2) = e(s_1, s_{3,0})$ and $e(s'_{2,1}, g_2) = e(s_1, s_{3,1})$ can both pass. The adversary $\mathcal{A}$ cannot guess the b with non-negligible probability because the tuple (s_1, s_2, s_3) meets the definition of IND. Therefore, we have

$$\Pr[\mathbf{Guess}_1] \leq \text{Adv}_{\mathcal{A}}^{\text{IND}}(\lambda)$$

- **Game₃**: This game is identical to game **Game₂** except that $\mathcal{S}$ outputs a random bit if a **Guess₂** event occurs in the g -th query. Here the **Guess₂** event means the probability of b' that the adversary guess for $\pi_{\sigma'}$ is equal to b . Then we have:

$$|\text{Adv}_2 - \text{Adv}_3| \leq \Pr[\mathbf{Guess}_2]$$

Lemma 2 The **Guess₂** event happens only with negligible probability when the **NIZKP** $\pi_{\sigma'}$ is zero-knowledge.

Let $\mathcal{S}$ denote an attacker, who is given a common random string crs and two proof generation oracles P_b as defined in Section 2.4, taking (e, w, crs) as input and outputting $\pi_{\sigma'}$. $\mathcal{S}$ aims to break **NIZKP**'s zero-knowledge property. Note that b is unknown to $\mathcal{S}$. $\mathcal{S}$ simulates the zero-knowledge game for $\mathcal{A}$ as follows.

- $\mathcal{S}$ sets up the game for $\mathcal{A}$ by creating n signers with the corresponding secret and public key pairs. Also, $\mathcal{S}$ creates an empty set $\mathcal{Q}$.
- At the challenge stage, if $\mathcal{A}$ submits a challenge holder $pk \in \mathcal{Q}$ and two messages (m_0, m_1) . Then, $\mathcal{S}$ performs the following steps. First, $\mathcal{S}$ chooses a statement e_0 that satisfies $\mathcal{R}(e_0, m_0) = 1$, forwards (e_0, m_0) to the proof generation oracle P_0 and receives a proof π_0 associated with the witness m_0 . $\mathcal{S}$ simulates another proof π_1 associated with the witness m_1 using the same method. Second, $\mathcal{S}$ simulates S_0 and S_1 using (sk, m_0) and (sk, m_1) based on the algorithm **Sign** defined in Section 3.1.2, respectively. Eventually, $\mathcal{S}$ returns one of two signatures $(\sigma'_0, \sigma'_1) = [(S_0, \pi_{\sigma'_0}), (S_1, \pi_{\sigma'_1})]$ to $\mathcal{A}$.

$\mathcal{S}$ outputs whatever $\mathcal{A}$ outputs. If $\mathcal{A}$ guesses the random bit correctly, $\mathcal{S}$ can break the **NIZKP**'s zero-knowledge property. Therefore, we have

$$\Pr[\mathbf{Guess}_2] \leq \text{Adv}_{\mathcal{S}}^{\text{ZK}}(\lambda)$$

Combining above results together, we have

$$\text{Adv}_{\mathcal{A}}(\lambda) \leq n \cdot q \cdot (\text{Adv}_{\mathcal{A}}^{\text{IND}}(\lambda) + \text{Adv}_{\mathcal{S}}^{\text{ZK}}(\lambda)).$$

The probability that $\mathcal{A}$ wins the message-hiding game is negligible, our proposed message-hiding scheme ensures message-hiding property.

Theorem 3 *The RMHS scheme in Section 3.1.2 is unforgeable if the PS signature is unforgeable and the **NIZKP** π_σ is simulation-sound extractable.*

Proof 3 We define a sequence of games Game_i , $i \in [0, 3]$ and let Adv_i denote the advantage of the adversary in game Game_i . In our unforgeability analysis, we provide our proof in the single-message case. Following the proof technique for handling multiple messages in [23, 24], our single message proof can be extended to the multi-message case. We assume that $\mathcal{A}$ issues at most q signing queries at each game.

- Game_0 : This is the original game for unforgeability.
- Game_1 : This game is identical to game Game_0 except that $\mathcal{S}$ randomly chooses $g \in [1, q]$ as a guess for the index of the forgery which returns a message-signature pair (m, σ') . $\mathcal{S}$ will output a random bit if $\mathcal{A}$'s forgery does not occur in this g -th query. Therefore, we have

$$\text{Adv}_0 = q \cdot \text{Adv}_1$$

- Game_2 : This game is identical to game Game_1 except that $\mathcal{S}$ will output a random bit if **Forge**₁ event happens where $\mathcal{A}$ outputs a forgery in the form of (m, σ') , such that the signature σ' is not returned by the message-hidden signing oracle. Then we have:

$$|\text{Adv}_1 - \text{Adv}_2| \leq \Pr[\text{Forge}_1]$$

Lemma 3 *The **Forge**₁ event happens only with negligible probability when the assumption 1 in definition 10 holds.*

Let $\mathcal{S}$ denote a solver to forge a PS signature, which is corresponding to a signing query, **Signing**, corresponding to **RMHS.Sign**. Given (g_2, X, Y) , $\mathcal{S}$ aims to compute (g_1, g_1^{x+my}) . $\mathcal{S}$ simulates the unforgeability game for $\mathcal{A}$ as follows:

- $\mathcal{S}$ sets up the game for $\mathcal{A}$ by creating a signature holder u and sets his public key as (g_2, X, Y) . $\mathcal{S}$ creates common random string crs and trapdoor information τ for the **NIZKP**.
- When $\mathcal{A}$ issues a signing query regards a statement e and a message (or witness) m_u , $\mathcal{S}$ forwards the message m_u to the signing oracle **Signing** and receives $(g_1, g_1^{x_u+m_u y_u})$. $\mathcal{S}$ first simulates π_{σ_u} using (crs, τ, e) when $\mathcal{R}(e, m_u) = 1$ (i.e., the witness relation is valid). Second, $\mathcal{S}$ simulates S_u as follows. $\mathcal{S}$ chooses two randomness $(r_{u,1}, r_{u,2})$, and computes $s_{u,1} = g_1^{r_{u,1}}$, $s_{u,2} = (g_1^{x_u+m_u y_u} \cdot g_1^{r_{u,2}})^{r_{u,1}}$, $s_{u,3} = g_2^{r_{u,2}} \cdot Y_u^{m_u}$. Eventually, $\mathcal{S}$ returns $(s_{u,1}, s_{u,2}, s_{u,3}, \pi_{\sigma_u})$ to $\mathcal{A}$.
- $\mathcal{A}$ outputs a forgery (m, S, e, π^*) , where $S = (s_1, s_2, s_3)$ and $s_1 = g_1^{r_1}$, $s_2 = (g_1^{x+my} \cdot g_1^{r_2})^{r_1}$, $s_3 = g_2^{r_2} \cdot Y^m$. $\mathcal{S}$ extracts the randomness r_2 from π^* due to its simulation-sound extractability. Then, $\mathcal{S}$ checks the following
 - * The **Forge**₁ event happens at g -th query;
 - * $S = (s_1, s_2, s_3)$ and the proof π^* for statement e are verified as valid;

* The forgery was not previously simulated by $\mathcal{S}$.

If all above conditions hold, $\mathcal{S}$ confirms that it is a successful forgery, $\mathcal{S}$ computes the solution $(g_1^{r_1}, g_1^{(x+my)r_1})$ to break the unforgeability of the PS signature. Therefore, we have

$$\Pr[\mathbf{Forge}_1] \leq \text{Adv}_S^{PS}(k)$$

- *Game₃*: This game is identical to game *Game₂* except that $\mathcal{S}$ will output a random bit if $\mathbf{Forge}_2$ event happens where $\mathcal{A}$ outputs a forgery in the form of (m, S, e, π^*) , where $S = (s_1, s_2, s_3)$, such that the proof (e, π^*) has not been returned by the SIM oracle in the message-hidden signature query, $\mathbf{RMHS.hSign}$, before. Then we have:

$$|\text{Adv}_2 - \text{Adv}_3| \leq \Pr[\mathbf{Forge}_2]$$

Lemma 4 The $\mathbf{Forge}_2$ event happens only with a negligible probability when the proof of knowledge system is simulation-sound extractable.

Let $\mathcal{S}$ denote an attacker against simulation-sound extractability property, who is given a common random string crs and a simulation oracle SIM, aims to output (e, π^*) , i.e., a proof π^* for a statement e associated with a witness m . $\mathcal{S}$ simulates the game for $\mathcal{A}$ as follows:

- $\mathcal{S}$ sets up the game for $\mathcal{A}$ by creating all signers' key pairs, and returns signers' public keys to $\mathcal{A}$. Note that crs is included in public keys.
- When $\mathcal{A}$ issues a signing query regards a statement e_u and a message m_u satisfying $\mathcal{R}(e_u, m_u) = 1$, $\mathcal{S}$ forwards e_u to his simulation oracle SIM and receives a proof π_{σ_u} associated with the witness m_u . Then, $\mathcal{S}$ simulates $S_u = (s_{u,1}, s_{u,2}, s_{u,3})$ using sk_u and m_u , and returns (S_u, π_{σ_u}) to $\mathcal{A}$.
- At some point, $\mathcal{A}$ outputs a forgery (m, S, e, π^*) . $\mathcal{S}$ checks: 1) the tuple $S = (s_1, s_2, s_3)$ is verified as valid; 2) the proof π^* for statement e is verified as valid; 3) (e, π^*) has not been returned by SIM oracle before. If all these conditions hold, $\mathcal{S}$ uses (e, π^*) to break the simulation-sound extractability property. Therefore, we have

$$\Pr[\mathbf{Forge}_2] \leq \text{Adv}_S^{Ext}(\lambda)$$

Combining the above results together, we have

$$\text{Adv}_{\mathcal{A}}(\lambda) \leq q \cdot (\text{Adv}_S^{PS}(\lambda) + \text{Adv}_S^{Ext}(\lambda)).$$

The probability that $\mathcal{A}$ wins the unforgeability game is negligible, therefore, our RMHS scheme is unforgeable.

3.2 Generic Construction

The generic construction of AVPEU consists of the building blocks **SIG**, **RMHS** and **NIZKP** defined in Section 2. The framework of the generic construction is shown in Figure. 5. An AVPEU scheme includes five algorithms/protocols, i.e., AVPEU = (KeyGen, VC.Gen, RVC.Gen, VP.Gen, VP.Verify), which are shown in Figure. 6.

- $\text{KeyGen}(\lambda) \rightarrow ((sk_n, pk_n), (sk_i, pk_i))$: Given a security parameter λ as input,
 1. The notary generates a secret-public key pair (sk_n, pk_n) by running $\text{RMHS.KeyGen}(\lambda)$. pk_n includes a **NIZKP** π_n , which is a proof of the corresponding key ownership sk_n .
 2. An issuer i generates a secret-public key pair (sk_i, pk_i) for an issuer i by running $\text{SIG.KeyGen}(\lambda)$. pk_i includes a **NIZKP** π_i , which is a proof of the corresponding key ownership, sk_i .
- $\text{VC.Gen}\{\lambda; sk_i, pk_i\} \rightarrow \{did_h, \mathbf{att}_h; cre_{h,i}\}$: This is a verifiable credential generation protocol run between a DID holder h and an issuer i .
 1. h verifies i 's key ownership, π_i . If invalid, aborts. Otherwise, h generates a secret/public key pair (sk_h, pk_h) , where pk_h acts as the holder's identity did_h . Then the holder requests a verifiable credential from the issuer i by proving the identity did_h using a **NIZKP** π_h .
 2. Then i verifies h 's proof, π_h . If the verification fails, aborts. Otherwise, i issues a verifiable credential $cre_{h,i}$ by running the algorithm SIG.Sign , $cre_{h,i} \leftarrow \text{SIG.Sign}(sk_i, (did_h, \mathbf{att}_h))$, where $\mathbf{att}_h$ is a set of attributes generated by the issuer for h .
 3. h verifies $cre_{h,i}$ by running $\text{SIG.Verify}(pk_i, (did_h, \mathbf{att}_h), cre_{h,i})$. If the verification fails, aborts; otherwise, accepts $cre_{h,i}$.
- $\text{RVC.Gen}\{cre_{h,i}, did_h, \mathbf{att}_h; sk_n, pk_n\} \rightarrow rcre_h$: This is a randomizable verifiable credential generation protocol run between a DID holder h and the notary n .
 1. h verifies the notary n by checking π_n . If invalid, aborts. Otherwise, sends a proof of his ownership of $cre_{h,i}$ to n .
 2. Then n verifies h 's proof of the ownership of $cre_{h,i}$. If fails, aborts; otherwise, accepts it and issues a $rcre_h$ for h by running the algorithm RMHS.Sign , $rcre_h \leftarrow \text{RMHS.Sign}(sk_n, (did_h, \mathbf{att}_h))$.
- $\text{VP.Gen}(pk_n, rcre_h, did_h, \mathbf{att}_h) \rightarrow vp_h$: This is a verifiable presentation generation algorithm run by h . h verifies $rcre_h$ by running the algorithm RMHS.Verify , $\text{RMHS.Verify}(pk_n, (did_h, \mathbf{att}_h), rcre_h)$. If the verification fails, aborts. Otherwise, accepts $rcre_h$. h generates a verifiable presentation by running the algorithm RMHS.hSign , $vp_h \leftarrow \text{RMHS.hSign}(rcre_h, (did_h, \mathbf{att}_h))$.
- $\text{VP.Verify}(pk_n, vp_h) \rightarrow 0/1$: This is a verifiable presentation verification algorithm run by the verifier, who takes (pk_n, vp_h) and runs $\text{RMHS.hVerify}(pk_n, vp_h)$.

Figure 6: The generic construction of AVPEU

3.2.1 Security Model and Analysis.

The security of an AVPEU scheme is captured by properties of unforgeability and anonymity. The correctness means an honestly computed VP can pass the verification.

Definition 11 (*Correctness*) An AVPEU scheme is correct, if

- $\text{KeyGen}(\lambda) \rightarrow ((sk_n, pk_n), (sk_i, pk_i)),$
- $\text{VC.Gen}\{\lambda; sk_i, pk_i\} \rightarrow \{did_h, \mathbf{att}_h; cre_{h,i}\},$
- $\text{RVC.Gen}\{cre_{h,i}, did_h, \mathbf{att}_h; sk_n, pk_n\} \rightarrow rcre_h$
- $\text{VP.Gen}(pk_n, rcre_h, did_h, \mathbf{att}_h) \rightarrow vp_h,$

then $\text{VP.Verify}(pk_n, vp_h) = 1.$

Theorem 4 Our generic construction of AVPEU offers correctness if **SIG**, **RMHS** and **NIZKP** (including π_n , π_i and π_h) are correct.

Proof 4 A valid holder can obtain a valid credential cre from an issuer because of the correctness of **SIG** and **NIZKP**. Using a valid credential cre , the holder can obtain a valid randomizable verifiable credential $rcre$ and a valid verifiable presentation vp , then vp can pass the verifiable presentation verification due to the correctness of **RMHS**.

Unforgeability. The unforgeability means that if issuers and the notary are all corrupted, the adversary cannot falsely attribute a VP to an honest holder who did not produce it. We define the unforgeability experiment in Figure. 7. The adversary can access to the following oracles, where **at** is a set of attributes that has been used to query to the oracle **RVCGen** and **VP** is a set of VPs returned by the oracle **VPGen**.

- **AddHU:** This oracle allows the adversary to act as an honest DID holder h to request a VC $cre_{h,i}$ from the issuer i . The adversary controls the issuer and the notary, gets their views. However, the adversary is not given the DID secret key, sk_h .
- **RVCGen:** This oracle allows the adversary to obtain a randomized verifiable credential $rcre_h$ under the honest DID holder's public key pk_h .
- **VPGen:** This oracle allows the adversary to obtain a VP vp_h under the honest holder's randomized credential $rcre_h$ and secret key sk_h .

Experiment $\text{Exp}_{\mathcal{A}}^{\text{unforge}}(\lambda)$

- $((sk_n, pk_n), (sk_i, pk_i)) \leftarrow \text{KeyGen}(\lambda)$
- $(vp^*, \mathbf{att}^*) \leftarrow \mathcal{A}^{\text{AddHU, RVCGen, VPGen}}(pp, sk_n, pk_n)$
- Return 1 iff $\text{VP.Verify}(pk_n, vp^*) = 1$ and $vp^* \notin \mathbf{VP}$ and $\mathbf{att}^* \notin \mathbf{at}$

Figure 7: The unforgeability experiment.

We denote the advantage of $\mathcal{A}$ as $\text{adv}_{\mathcal{A}}^{\text{unforge}}(\lambda)$, i.e., the probability that $\mathcal{A}$ wins the unforgeability game.

Definition 12 (*Unforgeability*) An AVPEU scheme is said to be unforgeable if for any adversary $\mathcal{A}$, $\text{adv}_{\mathcal{A}}^{\text{unforge}}$ is negligible.

$$\text{adv}_{\mathcal{A}}^{\text{unforge}} = |\Pr[\mathcal{A} \text{ wins}]|.$$

Theorem 5 Our generic construction of AVPEU offers unforgeability if **SIG** and **RMHS** are unforgeable, the **NIZKP** (including π_n , π_i and π_h) is a zero-knowledge proof system with simulation-sound extractability.

Proof 5 In the experiment, the adversary wins if it can generate (vp^*, att^*) , such that $\text{VP.Verify}(pk_n, vp^*) = 1$, and they were not generated by the honest holder.

Game₀: This game is the original unforgeability experiment.

Game₁: This game differs from *Game₀* only in that, when calling **AddHU**, **RVCGen** and **VPGen**, π_n , π_i and π_h are replaced by the output of their simulators. The advantage that the adversary can distinguish whether it is in *Game₀* or *Game₁* is negligible, otherwise, it contradicts the assumption that π_n , π_i and π_h are zero-knowledge. Therefore, we can get:

$$|\Pr[\text{Exp}_{\mathcal{A}}^{\text{unforge}}(\lambda) = 1] - \Pr[\text{Game}_1(\lambda) = 1]| \leq \text{neg}(\lambda)$$

The adversary outputs (vp^*, att^*) , there are four cases as follows to make the output of *Game₁* is 1:

- The adversary added a corrupted holder using a secret key $s_{h',1}$ of $\text{did}_{h'}$. Then generates a valid vp^* . Because of the unforgeability of the **RMHS**, we can get $s_{h',1} = s_{h^*,1}$, where $s_{h^*,1}$ is the secret key of did_{h^*} . Because the adversary cannot get the secret key $s_{h^*,1}$ of did_{h^*} by querying oracles or random guessing. If this case happens, it contradicts to the property of simulation-sound extractability of π_{h^*} .
- The adversary tries to forge a valid $\text{cre}_{h^*,i}$ to generate a valid vp^* without knowing the corresponding sk_i . Because $\text{cre}_{h^*,i}$ is a signature generated by **SIG**, if this case happens, it contradicts that **SIG** is unforgeable.
- The adversary tries to forge a valid randomizable verifiable credential rcre_{h^*} to generate a valid vp^* . Because rcre_{h^*} is a signature generated by **RMHS.Sign**, if this case happens, it contradicts that **RMHS** is unforgeable. Then we can get $sk_i = sk_{i^*}$. However the adversary cannot get sk_{i^*} by querying oracles or random guessing. If this case happens, it contradicts to the property of simulation-sound extractability of π_{i^*} .
- The adversary tries to forge a valid vp^* . If this case happens, it contradicts that **RMHS** is unforgeable.

Anonymity. The anonymity means that a VP does not reveal the identity of its holder, i.e., the adversary cannot distinguish which one of the two honest holders has signed a targeted message while both holders and the message are at the adversary's choice. The anonymity experiment is captured by $\text{Exp}_{\mathcal{A}}^{\text{anon}}(\lambda)$, run between an adversary $\mathcal{A}$ and a simulator $\mathcal{S}$, which is shown in Figure. 8.

Experiment $\text{Exp}_{\mathcal{A}}^{\text{Anon}}(\lambda)$

- $((sk_n, pk_n), (sk_i, pk_i)) \leftarrow \text{KeyGen}(pp, \lambda)$
- $b^* \leftarrow \mathcal{A}^{\text{AddHU, Corr, Chal, VPGen}}(sk_n, pk_n)$
- return 1 if $b^* = b$, where b is the index for the target holder.

Figure 8: The anonymity experiment

- **AddHU**: This oracle allows the adversary to act as multiple honest holders and honest issuers. The **VC.Gen** protocol is executed by the oracle and the adversary gets the honest holders' and issuers' views, i.e., holders' public keys, DIDs, and attributes and VCs generated by issuers and the notary's view, i.e., new randomizable credentials.
- **Corr**: This corrupt oracle allows the adversary to act as a corrupt holder. In this version, the issuer i and the notary n are corrupted. The adversary can get the corrupted parties' outputs and views.
- **Chal**: This oracle returns a challenge VP and can be called only once. The inputs to this oracle are a message m , two holders h_0 and h_1 . Then for $b \leftarrow_R \{0, 1\}$, did_{h_b} and att_{h_b} are used to generate a challenge VP vp_{h_b} , which is returned to the adversary.
- **VPGen**: This oracle allows the adversary to obtain a vp_h under the honest holder's randomizable credential rcr_{e_h} , DID did_h and attributes att_h .

$\mathcal{A}$ wins the anonymity game if $b^* = b$. The advantage of $\mathcal{A}$ is defined by

$$\text{adv}_{\mathcal{A}}^{\text{Anon}} = |\Pr[b^* = b] - 1/2|$$

Definition 13 (Anonymity). An AVPEU scheme offers anonymity if for any polynomial-time adversary $\mathcal{A}$, $\text{adv}_{\mathcal{A}}^{\text{Anon}}$ is negligible.

Theorem 6 The generic construction of AVPEU holds anonymity if the proof system (π_h) is a zero-knowledge proof system and the **RMHS** holds the message-hiding property.

Proof 6 In the anonymity game, the adversary needs to guess the DID holder to win the game. Consider the case that a vp will be returned during the challenge stage. In the following, we design a series of games to prove that the generic construction supports anonymity.

Game₀: This game is the original anonymity experiment.

Game₁: In this game, when calling the oracle **Chal**, vp is a signature of **RMHS**. Then the adversary cannot discover the identity of the signature holder (i.e., the DID holder). Otherwise, it contradicts the message-hiding property of **RMHS**.

Now, We need to ensure that the adversary knows nothing about sk_{h_b} . From the pseudocode of the oracles, we can conclude that the adversary cannot obtain sk_{h_b} directly. In particular:

- By calling **AddHU**, the returned output does not include sk_{h_b} of the honest holder directly;

- By calling **Corr**, the adversary can obtain secret keys of the corrupted holder. In the anonymity game, the adversary cannot nominate this corrupted holder in the challenge.
- The output of **VPGen** does not contain any information about sk_{h_b} .

While the adversary can not obtain sk_{h_b} directly, π_{h_b} can be seen by the adversary. Therefore, we have the next game:

Game₂: This game is modified from *Game₁*. In this game, π_{h_b} is replaced by the output of the simulator, i.e., $Sim_{\pi_{h_b}}$. We can say *Game₂* and *Game₁* are indistinguishable, otherwise, it contradicts the assumptions that π_{h_b} is zero-knowledge. After above 3 games, the final output during the challenge stage is vp_{h_b} . Therefore, the advantage that the adversary can distinguish all games from *Game₂* to *Game₀* is negligible. The generic construction of AVPEU holds anonymity.

3.3 Concrete Construction

In this subsection, we present a concrete construction of AVPEU in Figure. 9, which is based on the PS-type RMHS described in subsection 3.1.2. Due to the page limit, we defer CL and BBS-based concrete constructions to Appendix B and C, respectively. In Figure. 9, all public information is uploaded onto the verifiable data registry that each entity can obtain it. $\mathbb{G}_1$, $\mathbb{G}_2$ and $\mathbb{G}_T$ denote three cyclic groups of prime order p , $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, a type III bilinear map, a hash function $H : \{0,1\}^* \rightarrow \mathbb{Z}_p^*$. L denotes the number of attributes. Two generators for two groups are $g_1 \leftarrow_R \mathbb{G}_1^*$ and $g_2 \leftarrow_R \mathbb{G}_2^*$, respectively.

3.3.1 Security Analysis.

Here, we prove that the concrete construction of AVPEU holds correctness. For the unforgeability and anonymity, proofs are similar to the proofs of Theorem 5 and the Theorem 6, respectively. Therefore, we put proofs for unforgeability and anonymity in Appendix D and Appendix E, respectively, for readers who are interested in.

Theorem 7 *The proposed concrete construction of AVPEU is correct if $\pi_{h,1}$ offers completeness.*

Proof 7 *Given a verifiable presentation $vp_h = (\sigma'_1, \sigma'_2, \sigma'_3, \pi_{h,1})$, parse $\sigma'_1 = \sigma_1^{r_1}$, $\sigma'_2 = \sigma_1^{(r_2+x+\sum_{j=1}^L y_j \cdot s_{h,j}) \cdot r_1}$, $\sigma'_3 = g_2^{r_2} \cdot \prod_{j=1}^L Y_j^{s_{h,j}}$. We can get (1) $\sigma'_1 \neq 1_{\mathbb{G}_1}$, $\sigma'_2 \neq 1_{\mathbb{G}_1}$; and*

$$\begin{aligned} e(\sigma'_2, g_2) &= e(\sigma_1^{(r_2+x+\sum_{j=1}^L y_j \cdot s_{h,j}) \cdot r_1}, g_2) \\ &= e(\sigma_1^{r_1}, g_2^x \cdot g_2^{r_2} \cdot \prod_{j=1}^L Y_j^{s_{h,j}}) = e(\sigma'_1, X \cdot \sigma'_3) \end{aligned}$$

Due to the completeness of $\pi_{h,1}$, $\pi_{h,1}$ can pass the check. Therefore, the proposed concrete construction of AVPEU is correct.

- **KeyGen**(λ) $\rightarrow ((sk_n, pk_n), (sk_i, pk_i))$: Given the security parameter λ , The notary n selects secret keys $sk_n = (x, \{y_j\}) \leftarrow_R \mathbb{Z}_p^*$, $j \in [L]$ and computes $(X, \{Y_j\}) \leftarrow (g_2^x, g_2^{y_j})$, $j \in [L]$. Then generates a **NIZKP** π_n as follows:

$$\pi_n = \mathbf{NIZKP}\{\forall j \in [L], (g_2, X, \{Y_j\}); (x, \{y_j\}) | X = g_2^x, Y_j = g_2^{y_j}\}(m_n)$$

where m_n can be an empty string and the public key is $pk_n = (p, g_2, X, \{Y_j\}, \pi_n, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, H)$, $j \in [L]$. An issuer i generates a key pair (sk_i, pk'_i) by running **SIG.KeyGen**(λ) and a **NIZKP** π_i , which is similar to π_n , the public key $pk_i = (pk'_i, \pi_i)$.

- **VC.Gen** $\{\lambda; sk_i, pk_i\} \rightarrow \{did_h, \mathbf{att}_h; cre_{h,i}\}$: Given $(\lambda; sk_i, pk_i)$, h verifies π_i . If invalid, aborts. Otherwise, h , with $did_h = g_1^{s_{h,1}}$, where $s_{h,1} \leftarrow_R \mathbb{Z}_p^*$, sends a proof of the identity did_h to an issuer i using a **NIZKP** π_h , which is similar to π_n . i verifies π_h . If invalid, aborts. Otherwise, i runs $cre_{h,i} \leftarrow \mathbf{SIG.Sign}(sk_i, (did_h, \mathbf{att}_h))$, $\mathbf{att}_h = \{\mathbf{att}_{h,j}\}$ is a set of attributes generated by i for h , and $s_{h,j} \leftarrow_R \mathbb{Z}_p^*$, $\mathbf{att}_{h,j} = g_1^{s_{h,j}}$, $j \in \{2, \dots, L\}$. h verifies $cre_{h,i}$ by running **SIG.Verify**($pk_i, (did_h, \mathbf{att}_h), cre_{h,i}$). If the verification fails, aborts, otherwise, accepts $cre_{h,i}$.
- **RVC.Gen** $\{cre_{h,i}, did_h, \mathbf{att}_h; sk_n, pk_n\} \rightarrow rcre_h$: Given the input tuple $(cre_{h,i}, did_h, \mathbf{att}_h; sk_n, pk_n)$, h verifies π_n . If invalid, aborts. Else, h sends $(cre_{h,i}, did_h, \mathbf{att}_h)$ to n . n verifies h . If invalid, aborts. Otherwise, Selects $a \leftarrow_R \mathbb{Z}_p^*$ and computes $\sigma_1 = g_1^a$, σ_2 as follows:

$$\begin{aligned} \sigma_2 &= (g_1^a)^x \cdot \prod_{j=1}^L (\mathbf{att}_{h,j})^{a \cdot y_j} = (g_1^a)^x \cdot \prod_{j=1}^L (g_1^{s_{h,j}})^{a \cdot y_j} \\ &= (g_1^a)^x \cdot \prod_{j=1}^L (g_1^{y_j \cdot s_{h,j}}) = \sigma_1^{x + \sum_{j=1}^L y_j \cdot s_{h,j}} \end{aligned}$$

The final randomizable credential is $rcre_h = (\sigma_1, \sigma_2)$.

- **VP.Gen**($pk_n, rcre_h, did_h, \mathbf{att}_h$) $\rightarrow vp_h$: h firstly verifies $rcre_h$ by checking $e(\sigma_1, X \cdot \prod_{j=1}^L Y_j^{s_{h,j}}) \stackrel{?}{=} e(\sigma_2, g_2)$. If no, aborts; else, computes $\sigma'_1 = \sigma_1^{r_1}$, $\sigma'_2 = \sigma_1^{(r_2 + x + \sum_{j=1}^L y_j \cdot s_{h,j}) \cdot r_1}$, $\sigma'_3 = g_2^{r_2} \cdot \prod_{j=1}^L Y_j^{s_{h,j}}$, where $r_1, r_2 \leftarrow_R \mathbb{Z}_p^*$. Then generates a **NIZKP** $\pi_{h,1}$ as follows and $vp_h = (\sigma'_1, \sigma'_2, \sigma'_3, \pi_{h,1})$:

$$\pi_{h,1} = \mathbf{NIZKP}\{(pp, \sigma'_3); (r_2, s_{h,j})_{j \in [L]} | \sigma'_3 = g_2^{r_2} \cdot \prod_{j=1}^L Y_j^{s_{h,j}}\}(m)$$

- **VP.Verify**(pk_n, vp_h) $\rightarrow 0/1$: Given vp_h , the verifier checks: 1. $\sigma'_1 \neq 1_{\mathbb{G}_1}$, $\sigma'_2 \neq 1_{\mathbb{G}_1}$; 2. $e(\sigma'_2, g_2) = e(\sigma'_1, X \cdot \sigma'_3)$; 3. the validity of $\pi_{h,1}$. The verifier outputs “1” for accepting that vp_h . Otherwise, outputs “0” for rejecting it.

Figure 9: The PS-based AVPEU Scheme

Table 1: Complexity results of three instances

Algorithm	Party	Computational Cost	<i>rcre</i> Size	<i>vp</i> Size
CL-based instance in Appendix B				
VP.Gen	Holder	$(4 + 2L)\mathbb{G}_1 + 4P + (L + 2)\mathbb{G}_1^2$	$(4 + L) \mathbb{G}_1 + 2 \mathbb{Z}_p $	$(4 + L) \mathbb{G}_1 + (L + 1) \mathbb{Z}_p $
VP.Verify	Verifier	$1\mathbb{G}_1^L + 1\mathbb{G}_2^L + 6P + 1\mathbb{G}_1^{L+1}$		
BBS-based instance in Appendix C				
VP.Gen	Holder	$(4 + L)\mathbb{G}_1 + 4P + 1\mathbb{G}_2$	$1 \mathbb{G}_1 + 1 \mathbb{Z}_p $	$1 \mathbb{G}_1 + (L + 4) \mathbb{Z}_p $
VP.Verify	Verifier	$1\mathbb{G}_1^{L+3} + 2P + \mathbb{G}_1^2$		
PS-based instance in Section 3.3				
VP.Gen	Holder	$1 + \mathbb{G}_1 + 1\mathbb{G}_2^{L+1} + 1\mathbb{G}_2^L + 1\mathbb{G}_2 + 2P$	$2 \mathbb{G}_1 $	$2 \mathbb{G}_1 + 1 \mathbb{G}_2 + (L + 2) \mathbb{Z}_p $
VP.Verify	Verifier	$2P + 1\mathbb{G}_2^{L+2}$		

For the computational cost, $\mathbb{G}_i (i = \{1, 2, T\})$ denote cost of an exponentiation in the group $\mathbb{G}_i$; $\mathbb{G}_i^m$ denote the cost of a multi-exponentiation of m values in the group $\mathbb{G}_i$; P denote the cost of a pairing computation; For the credential size and verifiable presentation size, $|\mathbb{Z}_p|$ denote the size of an element in $\mathbb{Z}_p$, $|\mathbb{G}_i| (i = \{1, 2, T\})$ denote the size of an element of the group $\mathbb{G}_i$; L is the number of attributes.

4 Implementation and Evaluation

4.1 Complexity results

We first give complexity results for algorithms VP.Gen and VP.Verify of all three instances in Table 1. Because the key generation is only run by once, we omit it in the complexity table. In addition, the computational cost for the VC.Gen process depends on the signature algorithm **SIG**, the complexity for three instances are the same, therefore, we do not count the complexity for this process. For the process of RVC.Gen, the computational cost depends on concrete design of randomizable signature algorithm **rSIG**, i.e., CL, BBS and PS signatures. This comparison is obvious and trivial. We do not give the complexity results of RVC.Gen in these three instances. The most important difference is in VP.Gen and VP.Verify algorithms, which are given in the Table 1. From the Table 1, it is obvious that the credential size in the BBS-based instance and PS-based instance is constant, i.e., $1\mathbb{G}_1 + 1\mathbb{Z}_p$ and $2\mathbb{G}_1$, respectively. While the credential size in the CL-based instance is linear with L . Verifiable presentation size and computational cost in three instances are linear with L .

4.2 Implementation

Data availability statement:

The code for this manuscript is available at <https://github.com/Anonymous-Mystery/AVPEU>.

We have implemented three proposed instances of AVPEU, i.e., the CL-based instance and the BBS-based instance (in Appendix B and C, respectively), and the PS-based instance in section 3.3. Our experiments were run in Python 3.9.16 using the Charm 0.5 framework and the BN254 curve. All running times below were measured on a PC with a 3.59 GHz AMD Ryzen 5 3600 6-Core Processor and 16GB RAM.

These complexity results are tested in our implementation experi-

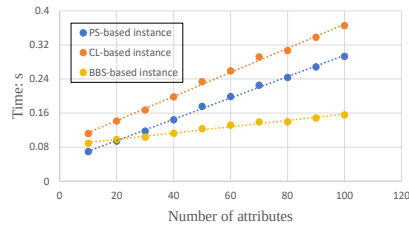
ments, the outcomes of which are given in Table 2, Table 3, and Figure 10. We run each experiment multiple times and compute the average running time. The average running time for various operations on this curve can be seen in Table 2. The average running time for choosing an element from $\mathbb{G}_1$ and $\mathbb{G}_2$ are 0.78 ms and 1.31 ms, respectively. An exponentiation in $\mathbb{G}_2$ (1.62 ms) is more expensive than that in $\mathbb{G}_1$ (0.74 ms). The pairing operation is 20.61 ms, which is much more expensive than other operations. The storage overhead for an element in each group can be seen in Table 3. The storage overhead in $\mathbb{G}_1$ and $\mathbb{G}_2$ are 0.2 KB and 0.36 KB, respectively, while a random number in $\mathbb{Z}_p$ is 0.12 KB.

Table 2: Average running time (ms) of operations

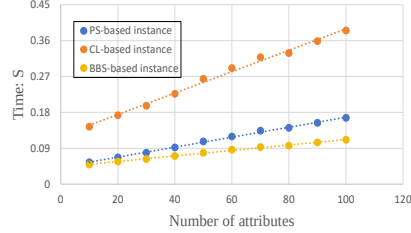
Groups	Choose	Exponentiation	Pairing
$\mathbb{G}_1$	0.78	0.74	20.61
$\mathbb{G}_2$	1.31	1.62	
$\mathbb{G}_T$	-	4.96	

Table 3: Storage overhead for an element in each group

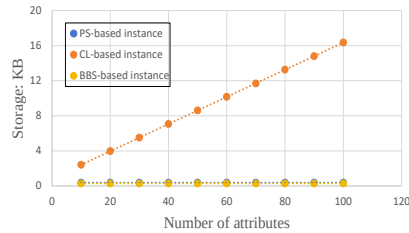
Elements in Groups	Size (KB)
g_1 in $\mathbb{G}_1$	0.20
g_2 in $\mathbb{G}_2$	0.36
g_t in $\mathbb{G}_T$	0.98
A random number in $\mathbb{Z}_p$	0.12



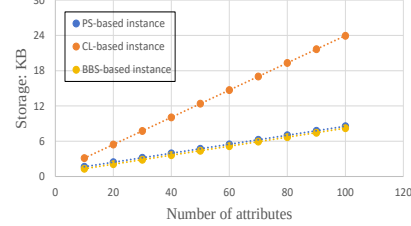
(a) Computational cost in VP.Gen algorithm



(b) Computational cost in VP.Verify algorithm



(c) Credential size



(d) VP size

Figure 10: Implementation results for three instances

For subfigures (a)-(d) in Figure 10, $L = \{10, 20, \dots, 100\}$. The subfigures (a)-(d) in Figure 10 show that the performance of CL-based instance is worse than the other two instances, because in the CL-based instance, there are multiple generators for both a holder and the notary.

Then, let us focus on the other two instances, i.e., the PS-based instance and the BBS-based instance. For the computational cost in the VP.Gen algorithm, when the number of a holder's attributes is less than 20, the PS-based instance is more efficient than the BBS-based instance. However, when the number of attributes is more than 20, the BBS-based instance is more efficient than the PS-based instance in terms of the computational cost in the VP.Gen algorithm. This is because in the VP.Gen algorithm, there are 4 pairings and $1\mathbb{G}_2$ scale multiplication in the BBS-based instance, and 2 pairings and $1\mathbb{G}_2^{L+1} + 1\mathbb{G}_2^L + 1\mathbb{G}_2$ scale multiplications in the PS-based instance. As we can see in Table 2, computational cost in $\mathbb{G}_2$ is more expensive than in $\mathbb{G}_1$, but the computational cost for pairings is the most expensive. Therefore, when L is small, the PS-based instance is more efficient than the BBS-based instance. In real applications, when the number of attribute is less than 20, the PS-based scheme is a more practical option compared with the BBS-based one. However, when L is larger than 20, the PS-based instance requires more computations than that in the BBS-based instance. For VP.Verify, it is obvious that there are more computations on $\mathbb{G}_2$ in the PS-based instance than that in the BBS-based instance. Therefore, the PS-based instance is less efficient than the BBS-based instance.

For the storage overhead, the credential size in these two instances are more or less the same, $2\mathbb{G}_1$ for the credential size in the PS-based instance and $1\mathbb{G}_1 + 1\mathbb{Z}_p$ for the credential size in the BBS-based instance. The verifiable presentation size in PS-based instance is also more or less the same as that in the BBS-based instance.

To conclude, the CL-based instance is the less efficient one compared with PS-based and BBS-based instances in terms of computational cost (VP.Gen and VP.Verify) and storage overhead (credential size and VP size). For computational cost, the PS-based instance performs better than the BBS-based one when the number of attributes is ≤ 20 . While the BBS-based one is better when the number of attributes is ≥ 20 . Finally, for the storage overhead, the PS-based instance is nearly the same as the BBS-based one.

5 Conclusion

In this paper, we introduced a generic construction of AVPEU. The unique feature of this construction is that the anonymous credentials can be correctly verified when the credentials' verification keys are not available in the mutually distributed domains. We also introduced a new cryptographic primitive called randomizable message-hiding signature. We presented concrete instantiations based on the well-known randomizable signatures including BBS, CL and PS. We provided detailed implementations and confirmed their practicality.

Acknowledgements

This work was supported by the European Union’s Horizon research and innovation program for support under grant agreement numbers: 779391 (FutureTPM), 952697 (ASSURED), 101019645 (SECANT), 101069688 (CONNECT), 101070627 (REWIRE) and 101095634 (ENTRUST). These projects are funded by the UK government’s Horizon Europe guarantee and administered by UKRI. We also thank the Chinese Scholarship Council (CSC) for supporting Yalan’s Ph.D. study.

References

- [1] Belenkiy, M., Camenisch, J., Chase, M., Kohlweiss, M., Lysyanskaya, A., and Shacham, H. (2009). Randomizable proofs and delegatable anonymous credentials. In *Proceedings of CRYPTO, Santa Barbara, CA, USA, August 16-20*, pages 108–125. Springer.
- [2] Blömer, J. and Bobolz, J. (2018). Delegatable attribute-based anonymous credentials from dynamically malleable signatures. In *Proceedings of ACNS, July 2-4, Leuven, Belgium*, pages 221–239. Cham: Springer International Publishing.
- [3] Boneh, D., Boyen, X., and Shacham, H. (2004). Short group signatures. In *Proceedings of CRYPTO, Santa Barbara, California, USA, August 15-19*, pages 41–55. Springer.
- [4] Bosk, D., Frey, D., Gestin, M., and Piolle, G. (2022). Hidden issuer anonymous credential. *Proceedings of PETs, Sydney, Australia and Online, July 11-15, 2022*:571–607.
- [5] Brunner, C., Gellersdörfer, U., Knirsch, F., Engel, D., and Matthes, F. (2020). DID and VC: Untangling decentralized identifiers and verifiable credentials for the web of trust. In *International Conference on Blockchain Technology and Applications, Honolulu, HI, USA, September 18-20*, pages 61–66.
- [6] Camenisch, J., Dubovitskaya, M., Lehmann, A., Neven, G., Paquin, C., and Preiss, F.-S. (2013). Concepts and languages for privacy-preserving attribute-based authentication. In *Policies and Research in Identity Management: Third IFIP WG 11.6 Working Conference, London, UK, April 8-9*, pages 34–52. Springer.
- [7] Camenisch, J. and Lysyanskaya, A. (2001). An efficient system for non-transferable anonymous credentials with optional anonymity revocation. In *Proceedings of EUROCRYPT, Innsbruck (Tyrol), Austria, May 6-10*, pages 93–118. Springer.
- [8] Camenisch, J. and Lysyanskaya, A. (2004). Signature schemes and anonymous credentials from bilinear maps. In *Proceedings of CRYPTO, Santa Barbara, California, USA, August 15-19*, pages 56–72. Springer.

- [9] Chen, L., Dong, C., El Kassem, N., Newton, C. J., and Wang, Y. (2023). Hash-based direct anonymous attestation. In *Proceedings of PQCrypto, Maryland, USA, August 16-18*, pages 565–600, Switzerland. Cham: Springer Nature.
- [10] Chen, L., Dong, C., El Kassem, N., Newton, C. J., and Wang, Y. (2024a). A new hash-based enhanced privacy ID signature scheme. In *Proceedings of PQCrypto, Oxford, UK, June 12-14*, pages 37–71, Switzerland. Cham: Springer Nature.
- [11] Chen, L., Dong, C., Newton, C. J., and Wang, Y. (2024b). Sphinx-in-the-head: group signatures from symmetric primitives. *ACM Transactions on Privacy and Security*, 27(1):1–35.
- [12] Chen, L., El Kassem, N., Lehmann, A., and Lyubashevsky, V. (2019). A framework for efficient lattice-based DAA. In *Proceedings of the 1st ACM Workshop on Cyber-Security Arms Race, London, UK, November 11 - 15*, pages 23–34.
- [13] Derler, D. and Slamanig, D. (2019). Key-homomorphic signatures: definitions and applications to multiparty signatures and non-interactive zero-knowledge. *Designs, Codes and Cryptography*, 87(6):1373–1413.
- [14] DIF (2020). Decentralized identity foundation. <https://identity.foundation/>.
- [15] Halpin, H. (2020). Vision: A critique of immunity passports and w3c decentralized identifiers. In *Security Standardisation Research, SSR, London, UK, November 30 – December 1*, pages 148–168. springer.
- [16] ID2020 (2020). Digital identity alliance. <https://id2020.org/>.
- [17] Kubach, M., Schunck, C. H., Sellung, R., and Roßnagel, H. (2020). Self-sovereign and decentralized identity as the future of identity management? *Open Identity Summit*.
- [18] Lux, Z. A., Thatmann, D., Zickau, S., and Beierle, F. (2020). Distributed-ledger-based authentication with decentralized identifiers and verifiable credentials. In *Conference on Blockchain Research and Applications for Innovative Networks and Services (BRAINS), 28-30 September, online*, pages 71–78. IEEE.
- [19] Maram, D., Malvai, H., Zhang, F., Jean-Louis, N., Frolov, A., Kell, T., Lobban, T., Moy, C., Juels, A., and Miller, A. (2021). CanDID: Can-do decentralized identity with legacy compatibility, sybil-resistance, and accountability. In *Proceedings of IEEE S&P, May 24-27, Online*, pages 1348–1366. IEEE.
- [20] Mühle, A., Grüner, A., Gayvoronskaya, T., and Meinel, C. (2018). A survey on essential components of a self-sovereign identity. *Computer Science Review*, 30:80–86.

- [21] Naik, N. and Jenkins, P. (2020). uPort open-source identity management system: An assessment of self-sovereign identity and user-centric data platform built on blockchain. In *International Symposium on Systems Engineering (ISSE)*, October 12 – November 12, Online, pages 1–7. IEEE.
- [22] Nottingham, M. (2020). URI Design and Ownership. RFC 8820.
- [23] Pointcheval, D. and Sanders, O. (2016). Short randomizable signatures. In *Proceedings of CT-RSA, San Francisco, CA, USA, February 29-March 4*, pages 111–126. Springer.
- [24] Pointcheval, D. and Sanders, O. (2018). Reassessing security of randomizable signatures. In *Proceedings of CT-RSA, San Francisco, CA, USA, April 16-20*, pages 319–338, Switzerland. Cham: Springer International.
- [25] Sanders, O. and Traoré, J. (2024). Compact issuer-hiding authentication, application to anonymous credential. *Proceedings of PETs, Bristol, UK, July 15–20, 2024*:645–658.
- [26] uPort (2020). uPort:open identity system for the decentralized web.
- [27] W3C (2018). Decentralized identifiers (DIDs) v0.11:data model and syntaxes for decentralized identifiers.
- [28] W3C (2023). Verifiable credentials data model v2.0. <https://www.w3.org/TR/vc-data-model-2.0/what-is-a-verifiable-credential>.

Appendix

A Definitions of properties in NIZKP

Firstly, there is a simulator $\text{SIM} = (\text{SIM}_1, \text{SIM}_2)$ used in following definitions of Zero-knowledge and Simulation-sound extractability properties.

Definition 14 (Correctness) An *NIZKP* Π is correct, if for λ , for all $\text{crs}_\Pi \leftarrow \text{Setup}(1^\lambda)$, for all $e \in \mathcal{L}$, for all w that $\mathcal{R}(e, w) = 1$, for all $\pi \leftarrow \text{Proof}(\text{crs}_\Pi, e, w)$, it holds that $\text{Verify}(\text{crs}_\Pi, e, \pi) = 1$.

Definition 15 (Zero-Knowledge) We define a formal experiment $\text{Adv}_\mathcal{A}^{\text{ZK}}(1^\lambda)$ between a PPT adversary $\mathcal{A}$ and a simulator $\text{SIM} = (\text{SIM}_1, \text{SIM}_2)$ in Figure. 11. Note that τ denotes a simulation trapdoor. We define the advantage of $\mathcal{A}$ as

$$\text{Adv}_\mathcal{A}^{\text{ZK}}(1^\lambda) = |\Pr[\text{Exp}_\mathcal{A}^{\text{ZK}}(1^\lambda) \rightarrow 1] - 1/2|.$$

An *NIZKP* Π for language $\mathcal{L}$ is zero-knowledge, if for any PPT $\mathcal{A}$, $\text{Adv}_\mathcal{A}^{\text{ZK}}(1^\lambda)$ is negligible in λ .

```

Experiment  $\text{Exp}_{\mathcal{A}}^{\text{ZK}}(1^\lambda)$ 
 $(\text{crs}_\Pi, \tau) \leftarrow \text{SIM}_1(1^\lambda), b \leftarrow \{0, 1\}$ 
 $b' \leftarrow \mathcal{A}^{P_b(\cdot, \cdot)}(\text{crs}_\Pi)$ 
  where  $P_0$  on input  $e, w$  :
    return  $\pi \leftarrow \text{Proof}(\text{crs}_\Pi, e, w)$ , if  $\mathcal{R}(e, w) = 1$ 
    return  $\perp$ 
  where  $P_1$  on input  $e, w$  :
    return  $\pi \leftarrow \text{SIM}_2(\text{crs}_\Pi, e, \tau)$ , if  $\mathcal{R}(e, w) = 1$ 
    return  $\perp$ 
if  $b' = b$ , return 1; else, return 0.

```

Figure 11: Zero-Knowledge.

```

Experiment  $\text{Exp}_{\mathcal{A}}^{\text{Sim}}(1^\lambda)$ 
 $(\text{crs}_\Pi, \tau, \rho) \leftarrow \text{Ext}_1(1^\lambda), \mathcal{Q} \leftarrow \emptyset$ 
 $(e^*, \pi^*) \leftarrow \mathcal{A}^{\text{SIM}(\cdot, \cdot)}(\text{crs}_\Pi)$ 
  where  $\text{SIM}$  on input  $e$  :
     $\pi \leftarrow \text{SIM}_2(\text{crs}_\Pi, e, \tau)$ 
     $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{(e, \pi)\}$ 
    return  $\pi$ 
 $w^* \leftarrow \text{Ext}_2(\text{crs}_\Pi, \rho, e^*, \pi^*)$ 
if  $1 = \text{Verify}(e^*, \pi^*) \wedge \mathcal{R}(e^*, w^*) = 0 \wedge (e^*, \pi^*) \notin \mathcal{Q}$ ,
  return 1;
else, return 0.

```

Figure 12: Simulation Sound Extractability.

Definition 16 (Simulation-sound Extractability) *We define a formal experiment*

$\text{Adv}_{\mathcal{A}}^{\text{Sim}}(1^\lambda)$ *run between a PPT adversary $\mathcal{A}$ and an extractor $\text{Ext} = (\text{Ext}_1, \text{Ext}_2)$ in Figure. 12. We define the advantage of $\mathcal{A}$ as*

$$\text{Adv}_{\mathcal{A}}^{\text{Sim}}(1^\lambda) = |\Pr[\text{Exp}_{\mathcal{A}}^{\text{Sim}}(1^\lambda) \rightarrow 1]|.$$

An NIZKP Π for language $\mathcal{L}$ is simulation-sound extractable, if for any PPT $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{\text{Sim}}(1^\lambda)$ is negligible in λ .

B CL-based instance of AVPEU

- **KeyGen** $(1^\lambda) \rightarrow ((sk_n, pk_n), (sk_i, pk_i))$: In this key generation algorithm, given the security parameter λ , let $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$ be three cyclic groups of prime order p and $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, which is a type III bilinear map; one extra hash function H is defined as $H : \{0, 1\}^* \rightarrow \mathbb{Z}_p^*$ used in **NIZKP**. Here L denotes the number of attributes, which can be handled in the DID and VC system. Selects the public group elements $g, g_1, \dots, g_L \in \mathbb{G}_1$ and $\tilde{g}, \tilde{g}_1, \dots, \tilde{g}_L \in \mathbb{G}_2$ randomly, where $g_j = g^{r_j}$ and $\tilde{g}_j = \tilde{g}^{r_j}$ and $r_j \in_R \mathbb{Z}_p^*$, for $j = \{1, \dots, L\} = [L]$. The notary selects (x, y) as $(x, y) \leftarrow_R \mathbb{Z}_p^*$ and computes $(\tilde{X} = \tilde{g}^x, \tilde{Y} = \tilde{g}^y)$. Then $sk_n = (x, y)$ and $pk_n = (p, g, \tilde{g}, \{g_j\}, \{\tilde{g}_j\}, \tilde{X}, \tilde{Y}, \pi_n, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, H)$, where π_n is a zero-knowledge proof as follows:

$$\pi_n = \text{NIZKP}\{(\tilde{g}, \tilde{X}, \tilde{Y}); (x, y) | \tilde{X} = \tilde{g}^x, \tilde{Y} = \tilde{g}^y\}$$

A secret key sk_i and a corresponding public information is pk'_i for an issuer i is generated by running **SIG.KeyGen**(λ). i also generates a **NIZKP** π_i , which is similar to π_n , for proving the key ownership and the final public key $pk_i = (pk'_i, \pi_i)$.

- **VC.Gen** $\{\lambda; sk_i, pk_i\} \rightarrow \{did_h, \mathbf{att}_h; cre_{h,i}\}$: This is a verifiable credential generation protocol run between a DID holder h and an issuer i . Firstly, h verifies π_i . If invalid, aborts. Otherwise, h , with the public key $pk_h = \prod_{j=1}^L g_j^{s_{h,j}}$, where $s_{h,j} \leftarrow_R \mathbb{Z}_p^*$, requests a verifiable credential from the issuer i by proving pk_h using a **NIZKP** π_h . Note that the first element in pk_h acts as the identity, i.e., $did_h = g_1^{s_{h,1}}$. Then i verifies h . If the verification fails, aborts. Otherwise, i issues a verifiable credential $cre_{h,i}$ by running $cre_{h,i} \leftarrow \mathbf{SIG.Sign}(sk_i, (did_h, \mathbf{att}_h))$, where $\mathbf{att}_h$ is a set of attributes generated by the issuer for h , i.e., $\mathbf{att}_h = \{att_{h,j}\}$, where $s_{h,j} \leftarrow_R \mathbb{Z}_p^*$, $att_{h,j} = g_j^{s_{h,j}}$, $j \in \{2, \dots, L\}$. h verifies $cre_{h,i}$ by running **SIG.Verify**($pk_i, (did_h, \mathbf{att}_h), cre_{h,i}$). If the verification fails, aborts, otherwise, accepts $cre_{h,i}$.
- **RVC.Gen** $\{cre_{h,i}, did_h, \mathbf{att}_h; sk_n, pk_n\} \rightarrow rcre_h$: This randomizable verifiable credential generation protocol is run between a holder h and the notary n . Firstly, h obtains π_n and verifies it. If invalid, aborts. Else, h sends the tuple $(cre_{h,i}, did_h, \mathbf{att}_h)$ to the notary. Then n verifies h . If invalid, aborts. Otherwise, selects a random number r from $\mathbb{Z}_p^*$ and computes a modified CL signature for the holder's public key pk_h as follows:

$$A = g^r; \quad B = A^y; \quad C = A^x \cdot pk_h^{rx y}; \quad D = pk_h^{ry}; \\ E_j = g_j^{ry}, \quad j \in [L]$$

Perform a **NIZKP** written as $(\hat{c}, \hat{s})$, which shows that the discrete logarithms to the base g of B , to the base g_j of E_j and to the base pk_h of D are equivalent:

$$w \in_R \mathbb{Z}_p^*; \quad \hat{c} = H(g^w \| g_1^w \| \dots \| g_L^w \| pk_h^w); \\ \hat{s} = w - \hat{c}ry$$

Finally, the randomizable credential is $rcre_h = (A, B, C, D, E_1, \dots, E_L, \hat{c}, \hat{s})$, then the notary sends it to the verifiable data registry.

- **VP.Gen**($pk_n, rcre_h, did_h, \mathbf{att}_h$) $\rightarrow vp_h$: This verifiable presentation generation algorithm is run by the holder h . h firstly verifies $rcre_h$ by checking two pairings as follows

$$e(A, \tilde{Y}) \stackrel{?}{=} e(B, \tilde{g}); \quad e(A \cdot D, \tilde{X}) \stackrel{?}{=} e(C, \tilde{g})$$

and

$$\hat{c} \stackrel{?}{=} H\left(B^{\hat{c}} \cdot g^{\hat{s}} \| E_1^{\hat{c}} \cdot g_1^{\hat{s}} \| \dots \| E_L^{\hat{c}} \cdot g_L^{\hat{s}} \| D^{\hat{c}} \cdot pk_h^{\hat{s}}\right)$$

If any one of above verifications fails, aborts; otherwise accepts it and computes as follows:

- Selects a random number a and computes

$$A' = aA; \quad B' = aB; \quad C' = aC; \quad D' = aD; \\ E'_j = aE_j, \quad j \in [L]$$

- For $j \in [L]$, computes $R_j = g_j^{w_j}$, w_j is a random number from $\mathbb{Z}_p^*$.
- Computes the hash value

$$c = H(A' || B' || C' || D' || E'_1 || \dots || E'_L || \prod_{j=1}^L R_j)$$

- For $j \in [L]$, computes $s_j = w_j - c \cdot s_{h,j}$;

The output $vp_h = (A', B', C', D', E'_1, \dots, E'_L, c, \{s_j\}), j \in [L]$.

- **VP.Verify**(pk_n, vp_h) $\rightarrow 0/1$: This verifiable presentation verification algorithm is run by a verifier, who firstly parses $vp_h (A', B', C', D', E'_1, \dots, E'_L, c, \{s_j\}), j \in [L]$ and checks as follows:

- $e(A', \tilde{Y}) \stackrel{?}{=} e(B', \tilde{g}), e(A' \cdot D', \tilde{X}) \stackrel{?}{=} e(C', \tilde{g})$.
- Verify the discrete logarithm equivalence as follows:

$$e(\prod_{j=1}^L E_j'^{t_j}, \tilde{g}) \stackrel{?}{=} e(B', \prod_{j=1}^L \tilde{g}_j^{t_j}), t_1, \dots, t_L \in_R \mathbb{Z}_p^*$$

- $\mu = c(D' - \prod_{j=1}^L (E_j')^{s_{h,j}})$.
- $c \stackrel{?}{=} H(A' || B' || C' || D' || E'_1 || \dots || E'_L)$.

Accept the verifiable presentation vp_h and outputs “1” if all the above verifications pass; otherwise reject it and outputs “0”.

C BBS-based instance of AVPEU

- **KeyGen**(1^λ) $\rightarrow ((sk_n, pk_n), (sk_i, pk_i))$: In this algorithm, given a security parameter λ as input, let $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$ be three cyclic groups of prime order p and $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, one extra hash function $H_1 : \{0, 1\}^* \rightarrow \mathbb{Z}_p^*$ is defined to be used in **NIZKP**. The notary selects the public group elements $g, g_0, g_1, \dots, g_L, H \in \mathbb{G}_1$ and $\tilde{g} \in \mathbb{G}_2$. The notary selects $x \leftarrow_R \mathbb{Z}_p^*$ and computes $\tilde{X} = \tilde{g}^x$. Finally, the notary's secret key is $sk_n = x$ and the public key is $pk_n = (p, g, g_0, \{g_j\}, H, \tilde{g}, \tilde{X}, \pi_n, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, H_1)$, $j \in [L]$, where π_n is a zero-knowledge proof as follows:

$$\pi_n = \mathbf{NIZKP}\{(\tilde{g}, \tilde{X}); x | \tilde{X} = \tilde{g}^x\}(m_n)$$

where m_n can be an empty string. A secret key sk_i and a corresponding public information is pk_i' for an issuer i is generated by running **SIG.KeyGen**(λ) for an issuer i . i also generates a **NIZKP** π_i , which is similar to π_n , for proving the key ownership and the final public key $pk_i = (pk_i', \pi_i)$.

- **VC.Cre**{ $\lambda; sk_i, pk_i$ } $\rightarrow \{did_h, \mathbf{att}_h; cre_{h,i}\}$: This is a verifiable credential generation protocol run between a DID holder h and an issuer i . Firstly, h verifies π_i . If invalid, aborts. Otherwise, h , with the public key $pk_h = \prod_{j=1}^L g_j^{s_{h,j}}$, where $s_{h,j} \leftarrow_R \mathbb{Z}_p^*$, requests a verifiable credential from the issuer i by proving pk_h using a **NIZKP** π_h . Note that the first element in pk_h acts as the identity, i.e., $did_h = g_1^{s_{h,1}}$. Then i verifies h .

If the verification fails, aborts. Otherwise, i issues a verifiable credential $cre_{h,i}$ by running $cre_{h,i} \leftarrow \mathbf{SIG.Sign}(sk_i, (did_h, \mathbf{att}_h))$, where $\mathbf{att}_h$ is a set of attributes generated by the issuer for h , i.e., $\mathbf{att}_h = \{att_{h,j}\}$, where $s_{h,j} \leftarrow_R \mathbb{Z}_p^*$, $att_{h,j} = g_j^{s_{h,j}}$, $j \in \{2, \dots, L\}$. h verifies $cre_{h,i}$ by running $\mathbf{SIG.Verify}(pk_i, (did_h, \mathbf{att}_h), cre_{h,i})$. If the verification fails, aborts, otherwise, accepts $cre_{h,i}$.

- **RVC.Gen** $\{cre_{h,i}, did_h, \mathbf{att}_h; sk_n, pk_n\} \rightarrow rcre_h$: This randomizable verifiable credential generation protocol is run between a holder h and the notary. Firstly, h obtains π_n and verifies it. If invalid, aborts. Else, h sends the tuple $(cre_{h,i}, did_h, \mathbf{att}_h)$ to the notary. Then n verifies h . If invalid, aborts. Otherwise, h selects a random number ξ and compute a sDH signature $rcre_h$:

$$\xi \in_R \mathbb{Z}_p^*; \quad A := (g_0 \cdot pk_h)^{\frac{1}{(x+\xi)}}; \quad rcre_h = (A, \xi).$$

Then the notary sends it to the verifiable data registry.

- **VP.Gen** $(pk_n, rcre_h, did_h, \mathbf{att}_h) \rightarrow vp_h$: This verifiable presentation generation algorithm is run by the holder h . h firstly verifies $rcre_h$ by checking $e(A, \tilde{X} \cdot \tilde{g}^\xi) \stackrel{?}{=} e(g_0 \cdot pk_h, \tilde{g})$. If no, aborts; otherwise accepts it and computes as follows:
 - selects $\eta \in_R \mathbb{Z}_p^*$ and computes $A' := A \cdot \eta H$.
 - For $j \in [L]$, compute $R_j = g_j^{w_j}$, w_j is a random number from $\mathbb{Z}_p^*$.
 - Computes the hash value.

$$w_\alpha, w_\beta, w_\gamma \in_R \mathbb{Z}_q; \mu := e(H^{w_\alpha}, \tilde{X});$$

$$\omega := e\left((A')^{w_\beta} \cdot H^{w_\gamma} \cdot \prod_{j=1}^L R_j, \tilde{g}\right)$$

$$c := H_1(\mu\omega)$$

$$s_\alpha := w_\alpha - c\eta; \quad s_\beta := w_\beta + c\xi; \quad s_\gamma := w_\gamma - c\eta\xi;$$

- For $j \in [L]$ computes

$$s_j = w_j - c \cdot s_{h,j}$$

- The final verifiable presentation vp_h

$$vp_h = (A', c, s_\alpha, s_\beta, s_\gamma, \{s_j\}), \quad j \in [L]$$

Then the holder sends the verifiable presentation vp_h to the verifier.

- **VP.Verify** $(pk_n, vp_h, m) \rightarrow 0/1$: This verifiable presentation verification algorithm is run by a verifier, who parses vp_h as $(A', c, s_\alpha, s_\beta, s_\gamma, \{s_j\})$, $j \in [L]$, and checks as follows:

$$\mu' := e((A') \cdot H^{s_\alpha}, \tilde{X})$$

$$\omega' := e\left((A')^{s_\beta} \cdot H^{s_\gamma} / g_0^c \cdot \prod_{j \in [L]} g_j^{s_j}, \tilde{g}\right)$$

$$c \stackrel{?}{=} H_1(\mu' \cdot \omega')$$

Accept the verifiable presentation vp_h and outputs “1” if all the above verifications pass; otherwise reject it and outputs “0”.

D Security proof of unforgeability

Theorem 8 *The proposed concrete construction of AVPEU is unforgeable if **SIG** and **RMHS** are unforgeable, and the **NIZKP** (including π_n , π_i , π_h) is a zero-knowledge proof system with simulation-sound extractability.*

Proof 8 *Following the security model in Section 3.2.1, an adversary wins the unforgeability game if it can generate $(vp^*, \mathbf{att}^*)$, such that $VP.Verify(pk_n, vp^*) = 1$, and this tuple was not generated by the honest holder.*

Game₀: This game is the original unforgeability experiment.

*Game₁: This game differs from Game₀ only in that, when calling **AddHU**, **RVCGen** and **VPGen**, π_n , π_i and π_h are replaced by the output of their simulators. The advantage that the adversary can distinguish whether it is in Game₀ or Game₁ is negligible, otherwise, it contradicts the assumption that π_n , π_i and π_h are zero-knowledge. Therefore, we can get:*

$$|Pr[Exp_A^{unforge}(\lambda) = 1] - Pr[Game_1(\lambda) = 1]| \leq neg(\lambda)$$

The adversary outputs $(vp^, \mathbf{att}^*)$, there are four cases as follows to make the output of Game₁ is 1:*

- *The adversary added a corrupted holder using a secret key $s_{h',1}$ of $did_{h'}$. Then generates a valid verifiable presentation vp^* . Because vp^* is an **RMHS** signature, which is unforgeable, we can get $s_{h',1} = s_{h^*,1}$, where $s_{h^*,1}$ is the secret key of did_{h^*} . Because the adversary cannot get the secret key $s_{h^*,1}$ of did_{h^*} by querying oracles or random guessing. If this case happens, due to the property of simulation-sound extractability of π_{h^*} , the secret key $s_{h^*,1}$ can be extracted. This case happens with negligible probability.*
- *The adversary tries to forge a valid verifiable credential $cre_{h^*,i}$ to generate a valid vp^* without knowing the corresponding sk_i . Because $cre_{h^*,i}$ is a signature generated by **SIG**, if this case happens, it contradicts that **SIG** is unforgeable.*
- *The adversary tries to forge a valid randomizable verifiable credential $rcre_{h^*}$ to generate a valid vp^* . Because $rcre_{h^*}$ is a signature generated by **RMHS.Sign**, if this case happens, it contradicts that **RMHS** is unforgeable. Then we can get $sk_i = sk_{i^*}$. However the adversary cannot get sk_{i^*} by querying oracles or random guessing. If this case happens, due to the property of simulation-sound extractability of π_{i^*} , sk_{i^*} can be extracted. This case happens with negligible probability.*
- *The adversary tries to forge a valid vp^* . If this case happens, it contradicts that **RMHS** is unforgeable.*

E Security proof of anonymity

Theorem 9 *The concrete construction of AVPEU holds anonymity if the proof system (π_h) is a zero-knowledge proof system and the **RMHS** holds the message-hiding property.*

Proof 9 In the anonymity game, the adversary needs to guess the DID holder to win the game. Consider the case that a vp will be returned during the challenge stage. In the following, we design a series of games to prove that the generic construction supports anonymity.

Game₀: This game is the original anonymity experiment.

Game₁: In this game, when calling the oracle **Chal**, vp is a signature of **RMHS**. Then the adversary cannot discover the identity of the signature holder (i.e., the DID holder). Otherwise, it contradicts the message-hiding property of **RMHS**.

Now, We need to ensure that the adversary knows nothing about sk_{h_b} . From the pseudocode of the oracles, we can conclude that the adversary cannot obtain sk_{h_b} directly. In particular:

- By calling **AddHU**, the returned output does not include sk_{h_b} of the honest holder directly;
- By calling **Corr**, the adversary can obtain secret keys of the corrupted holder. In the anonymity game, the adversary cannot nominate this corrupted holder in the challenge.
- The output of **VPGen** does not contain any information about sk_{h_b} .

While the adversary can not obtain sk_{h_b} directly, π_{h_b} can be seen by the adversary. Therefore, we have the next game:

Game₂: This game is modified from *Game₁*. In this game, π_{h_b} is replaced by the output of the simulator, i.e., $Sim_{\pi_{h_b}}$. We can say *Game₂* and *Game₁* are indistinguishable, otherwise, it contradicts the assumptions that π_{h_b} is zero-knowledge. After above 3 games, the final output during the challenge stage is vp_{h_b} . Therefore, the advantage that the adversary can distinguish all games from *Game₂* to *Game₀* is negligible. The generic construction of AVPEU holds anonymity.